

**ACCESS - ADAPTIVE COGNITIVE AND EDUCATIONAL
SKILL SUPPORT**

MUHAMMAD ALIFF BIN AFFANDI

UNIVERSITI TEKNIKAL MALAYSIA MELAKA

ACCESS - ADAPTIVE COGNITIVE AND EDUCATIONAL SKILL SUPPORT

MUHAMMAD ALIFF BIN AFFANDI

This report is submitted in partial fulfillment of the requirements for the
Bachelor of Computer Science (Software Development) with Honours.

FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY
UNIVERSITI TEKNIKAL MALAYSIA MELAKA

2026

DECLARATION

I hereby declare that this project entitled
ACCESS - ADAPTIVE COGNITIVE AND EDUCATIONAL SKILL SUPPORT
is written by me and is my own effort and that no part has been plagiarized without
citations.

STUDENT : _____ Date : _____
(MUHAMMAD ALIFF BIN AFFANDI)

I hereby declare that I have read this project report and found this project report is
sufficient in term of the scope and quality for the award of Bachelor of Computer
Science (Software Development) with Honours.

SUPERVISOR : _____ Date : _____
(TS. DR. ZURAINI BINTI OTHMAN)

DEDICATION

To my beloved parents, whose endless sacrifices and unwavering faith have been my foundation through every challenge. To Ts. Dr. Zuraini Binti Othman, whose mentorship and invaluable guidance shaped not only this research but also my growth as a researcher and software developer. To all neurodivergent learners who navigate a world not yet built for them, may ACCESS serve as a small step toward a future where educational platforms adapt to every mind rather than demanding that every mind adapt to them.

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to my supervisor, Ts. Dr. Zuraini Binti Othman, for her continuous guidance, patience, and insightful feedback throughout the development of this project. Her expertise in software engineering and dedication to accessibility has been instrumental in shaping the ACCESS platform into a functional and meaningful system.

My heartfelt thanks go to my family and friends, who offered unwavering emotional and moral support during the challenging moments of this journey. Their encouragement was a constant source of motivation during late nights and difficult debugging sessions.

ABSTRACT

Traditional e-learning platforms are predominantly designed with a one-size-fits-all approach, presenting significant accessibility barriers for neurodivergent learners, particularly those with Dyslexia, Attention Deficit Hyperactivity Disorder (ADHD), and Autism Spectrum Disorder (ASD). These systems typically feature dense typography, complex navigational structures, and rigid content delivery that contribute to cognitive overload and high dropout rates. This project presents the development of the Adaptive Cognitive and Educational Skill Support (ACESS) platform, a web-based learning management system engineered to dynamically accommodate diverse cognitive profiles through natively integrated accessibility features. The system was developed using the Agile Scrum methodology over six iterative sprints, employing Next.js with the App Router for server-rendered React components, Supabase for backend-as-a-service with PostgreSQL Row-Level Security, and Tailwind CSS v4 for dynamic theme injection. ACESS implements core features including a rule-based adaptive engine that automatically recommends remedial content based on assessment performance, configurable accessibility profiles that alter typography, spacing, and motion preferences in real time via CSS custom properties, a strict Focus Mode for learners with attention-related challenges, and comprehensive analytics dashboards for educators to monitor engagement and identify at-risk students. The evaluation methodology comprised structured walkthroughs guided by task scenarios for each learner profile (Dyslexia, ADHD, ASD), complemented by an expert review conducted against WCAG 2.1 Level AA success criteria and established usability heuristics to systematically assess the platform's accessibility compliance. These evaluations demonstrated that the adaptive engine and accessibility settings effectively reduce cognitive friction and accommodate diverse learner needs compared to conventional platforms. The findings indicate that a purpose-built, architecturally adaptive system can significantly reduce cognitive friction and promote equitable educational outcomes for learners with diverse neurological needs.

ABSTRAK

Platform e-pembelajaran tradisional kebanyakannya direka dengan pendekatan satu-saiz-untuk-semua, yang menimbulkan halangan kebolehcapaian yang signifikan bagi pelajar neurodivergen, terutamanya mereka yang mengalami Disleksia, Attention Deficit Hyperactivity Disorder (ADHD), dan Autism Spectrum Disorder (ASD). Sistem-sistem ini biasanya menampilkan tipografi yang padat, struktur navigasi yang kompleks, dan penyampaian kandungan yang tegar, yang menyumbang kepada beban kognitif dan kadar keciciran yang tinggi. Projek ini membentangkan pembangunan platform Adaptive Cognitive and Educational Skill Support (ACESS), sebuah sistem pengurusan pembelajaran berasaskan web yang direka untuk menampung profil kognitif yang pelbagai secara dinamik melalui ciri kebolehcapaian yang diintegrasikan secara natif. Sistem ini dibangunkan menggunakan metodologi Agile Scrum melalui enam lalaran sprint, menggunakan Next.js dengan App Router untuk komponen React yang dipaparkan di pelayan, Supabase sebagai backend-as-a-service dengan keselamatan peringkat baris PostgreSQL, dan Tailwind CSS v4 untuk suntikan tema dinamik. ACESS melaksanakan ciri teras termasuk enjin adaptif berasaskan peraturan yang secara automatik mengesyorkan kandungan pemulihan berdasarkan prestasi penilaian, profil kebolehcapaian boleh konfigurasi yang mengubah tipografi, jarak, dan keutamaan gerakan secara masa nyata melalui pemboleh ubah CSS, mod Fokus yang ketat untuk pelajar yang mempunyai cabaran berkaitan perhatian, serta papan pemuka analitik komprehensif untuk pendidik memantau penglibatan dan mengenal pasti pelajar berisiko. Penilaian melalui huraian berstruktur berdasarkan senario tugas untuk setiap profil pembelajaran (Disleksia, ADHD, ASD), dilengkapi dengan semakan pakar yang dijalankan berdasarkan kriteria kejayaan WCAG 2.1 Peringkat AA dan heuristik kebolehgunaan yang ditetapkan untuk menilai pematuhan kebolehcapaian platform secara sistematik, menunjukkan bahawa enjin adaptif dan tetapan kebolehcapaian berkesan mengurangkan geseran kognitif dan memenuhi keperluan pelajar yang pelbagai berbanding platform konvensional. Dapatan kajian menunjukkan bahawa sistem yang dibina secara adaptif dari segi seni bina dapat mengurangkan geseran kognitif dengan ketara dan menggalakkan hasil pendidikan yang saksama untuk pelajar dengan keperluan neurologi yang pelbagai.

TABLE OF CONTENTS

	PAGE
DECLARATION	ii
DEDICATION	iii
ACKNOWLEDGEMENT	iv
ABSTRACT	v
ABSTRAK	vi
TABLE OF CONTENTS	vii
LIST OF TABLES	x
LIST OF FIGURES	xii
LIST OF ABBREVIATIONS	xiii
CHAPTER 1: INTRODUCTION	1
1.1 Background of Study	1
1.2 Problem Statement	2
1.3 Objectives	3
1.4 Scope	3
1.4.1 Target Users	3
1.4.2 Core System Modules	4
1.5 Significance of Project	5
1.6 Expected Output	6
1.7 Conclusion	7
CHAPTER 2: LITERATURE REVIEW AND PROJECT METHODOLOGY	8
2.1 Introduction	8
2.2 Theoretical Frameworks for Cognitive Accessibility	8
2.2.1 Universal Design for Learning (UDL)	9
2.2.2 Web Content Accessibility Guidelines (WCAG) 2.2	9

2.3	Neurodivergent Profiles and Interface Interventions	10
2.3.1	Attention Deficit Hyperactivity Disorder (ADHD)	10
2.3.2	Dyslexia	11
2.3.3	Autism Spectrum Disorder (ASD)	11
2.4	Comparative Analysis of Existing LMS Platforms	12
2.4.1	Moodle	12
2.4.2	Canvas LMS	12
2.4.3	Blackboard Learn	13
2.4.4	Google Classroom	13
2.5	Comparison Matrix	13
2.5.1	Alternative Approaches and Justification	14
2.6	Project Methodology	15
2.7	Project Requirements	16
2.7.1	Software Requirements	16
2.7.2	Hardware Requirements	18
2.7.3	Other Requirements	18
2.8	Project Schedule and Milestones	19
2.9	Summary	20
CHAPTER 3: SYSTEM ANALYSIS		21
3.1	Introduction	21
3.2	Problem Analysis	21
3.2.1	Current System Scenario	21
3.2.2	Detailed Problem Breakdown	22
3.3	Requirement Analysis	23
3.3.1	Data Requirement	23
3.3.2	Functional Requirement	24
3.3.3	Non-Functional Requirement	28
3.3.4	Others Requirement	30
3.4	System Architecture	31
3.4.1	Context Diagram	31
3.4.2	System Architecture Diagram	33
3.4.3	Deployment Diagram	35

3.5	Module Design	36
3.6	Summary	36
CHAPTER 4: SYSTEM DESIGN		38
4.1	Introduction	38
4.2	High-Level Design	38
4.2.1	Behavioral Design (Activity Diagrams)	38
4.2.2	Interaction Design (Sequence Diagrams)	45
4.2.3	User Interface Design	49
4.2.4	Database Design & Entity Relationship	52
4.3	Comprehensive Data Dictionary	55
4.3.1	Core Identity and Configuration	55
4.3.2	Curriculum Structure	60
4.3.3	Telemetry and Assessment	67
4.3.4	Learner Progress and Checkpoints	70
4.3.5	Content and Media	71
4.4	Detailed Design	77
4.4.1	Software Design	77
4.4.2	Physical Database Design	83
4.4.3	Systemic Implementation Details	84
4.5	Limitations of Design	85
4.6	Summary	86
References		87

LIST OF TABLES

	PAGE
Table 2.1 LMS Feature Comparison Matrix	14
Table 2.2 Project Schedule and Milestones	19
Table 3.1 Functional Requirements: Student Module	26
Table 3.2 Functional Requirements: Educator Module	27
Table 3.3 Functional Requirements: Admin Module	27
Table 4.1 Users Data Dictionary	55
Table 4.2 User Profiles Data Dictionary	56
Table 4.3 User Accessibility Preferences Data Dictionary	57
Table 4.4 Enrollments Data Dictionary	57
Table 4.5 Accessibility Templates Data Dictionary	58
Table 4.6 Adaptive Interactions Data Dictionary	58
Table 4.7 Instructor Applications Data Dictionary	59
Table 4.8 Password Reset Tokens Data Dictionary	60
Table 4.9 Referral Codes Data Dictionary	60
Table 4.10 Courses Data Dictionary	61
Table 4.11 Lessons Data Dictionary	63
Table 4.12 Quizzes Data Dictionary	64
Table 4.13 Quiz Questions Data Dictionary	64
Table 4.14 Quiz Options Data Dictionary	65
Table 4.15 Course Chapters Data Dictionary	65
Table 4.16 Course Milestones Data Dictionary	66
Table 4.17 Course Achievements Data Dictionary	66
Table 4.18 Course Accessibility Categories Data Dictionary	67
Table 4.19 Course Favorites Data Dictionary	67

Table 4.20 Quiz Attempts Data Dictionary	67
Table 4.21 Quiz Answers Data Dictionary	68
Table 4.22 Lesson Progress Data Dictionary	68
Table 4.23 Lesson Checkpoints Data Dictionary	69
Table 4.24 Learner Checkpoints Data Dictionary	69
Table 4.25 Learner Milestones Data Dictionary	70
Table 4.26 Video Questions Data Dictionary	70
Table 4.27 Recommendations Data Dictionary	71
Table 4.28 User Achievements Data Dictionary	71
Table 4.29 Lesson Comments Data Dictionary	72
Table 4.30 Lesson Summaries Data Dictionary	72
Table 4.31 Lesson Versions Data Dictionary	73
Table 4.32 Lesson Interactive Content Data Dictionary	73
Table 4.33 Lesson Templates Data Dictionary	74
Table 4.34 Media Assets Data Dictionary	74
Table 4.35 Notifications Data Dictionary	75
Table 4.36 Contact Messages Data Dictionary	75
Table 4.37 Certificates Data Dictionary	76
Table 4.38 Certificate Templates Data Dictionary	77
Table 4.39 Certificate Verifications Data Dictionary	77

LIST OF FIGURES

	PAGE
Figure 2.1 Gantt chart of ACESS development schedule (14 weeks)	20
Figure 3.1 Current System Problem Analysis Diagram	22
Figure 3.2 Use Case Diagram for ACESS Platform	25
Figure 3.3 Context Diagram	32
Figure 3.4 System Architecture Diagram	33
Figure 3.5 Deployment Diagram	35
Figure 4.1 Student Activity Diagram	40
Figure 4.2 Educator Activity Diagram	43
Figure 4.3 Administrator Activity Diagram	45
Figure 4.4 Course Enrollment Sequence Diagram	46
Figure 4.5 Lesson Learning Sequence Diagram	47
Figure 4.6 Quiz Attempt Sequence Diagram	48
Figure 4.7 Course Creation Sequence Diagram	49
Figure 4.8 Database Entity Relationship Diagram (Full resolution version provided in Appendix)	54

LIST OF ABBREVIATIONS

ACCESS	- Adaptive Cognitive and Educational Skill Support
ADHD	- Attention Deficit Hyperactivity Disorder
API	- Application Programming Interface
ASD	- Autism Spectrum Disorder
CSS	- Cascading Style Sheets
ERD	- Entity Relationship Diagram
HTML	- Hypertext Markup Language
JSON	- JavaScript Object Notation
JWT	- JSON Web Token
LMS	- Learning Management System
OOAD	- Object-Oriented Analysis and Design
PDF	- Portable Document Format
PostgreSQL	- Object-Relational Database Management System
PSM	- Proyek Sarjana Muda
RBAC	- Role-Based Access Control
React	- JavaScript Library for Building User Interfaces
RLS	- Row-Level Security
Supabase	- Open-Source Backend-as-a-Service
Tailwind	- Utility-First CSS Framework
Tiptap	- Headless Rich-Text Editor
TTS	- Text-to-Speech
UDL	- Universal Design for Learning
UI	- User Interface
URL	- Uniform Resource Locator
Vercel	- Cloud Platform for Frontend Deployment
WCAG	- Web Content Accessibility Guidelines

CHAPTER 1: INTRODUCTION

1.1 Background of Study

In the current educational landscape, instruction is delivered through a blend of traditional face-to-face classroom settings and online platforms [Prensky, 2001]. While digital tools have become increasingly common to support learning, many institutions still rely heavily on traditional teaching methods. This hybrid approach presents unique challenges for neurodivergent individuals, specifically those with Dyslexia, Attention Deficit Hyperactivity Disorder (ADHD), and Autism Spectrum Disorder (ASD). Whether learning occurs in a physical classroom or online, the tools used to deliver content often lack the flexibility required to accommodate diverse cognitive profiles.

Existing Learning Management Systems (LMS) are typically designed with a “one-size-fits-all” approach. They present dense textual information, complex navigation structures, and rigid assessment formats. These standard interfaces can erect significant barriers for neurodivergent learners. For instance, learners with Dyslexia may struggle with standard typography and line spacing [Franceschini et al., 2013], while those with ADHD might find cluttered interfaces and unstructured learning paths overwhelming [Barkley, 1997].

Therefore, there is a critical need for a natively adaptive learning platform that can seamlessly support both face-to-face and remote learning environments. The Adaptive Cognitive and Educational Skill Support (ACESS) platform is proposed to bridge this gap. ACESS is engineered to provide an inclusive digital learning environment that dynamically alters its interface and content delivery based on the specific cognitive requirements of the user. By integrating features such as dynamic typography

adjustment, focus-driven layouts, and rule-based adaptive learning paths, ACCESS aims to ensure that educational materials are accessible and equitable for all learners, regardless of the instructional setting.

1.2 Problem Statement

Despite the availability of various educational technologies, significant cognitive accessibility barriers persist. The core problem this project addresses is the systemic inadequacy of current educational platforms to serve learners with disabilities.

Specifically, the problems identified are:

1. Most existing e-learning platforms are not designed for learners with disabilities. They use formats that overwhelm cognitive and sensory processing, leading to high dropout rates. Research indicates that learners with disabilities face significantly higher attrition rates in online education; for instance, only 46% of neurodivergent students completed online courses compared to 74% of their neurotypical peers [Spencer and Podesta, 2022], while up to 35% of students with ADHD drop out of e-learning programs within the first semester due to poor accessibility design [Burl and O'Brien, 2024].
2. Learners with disabilities lack platforms that adapt to their individual learning pace and present content in accessible, easy-to-process formats.
3. Employers and support organisations have no reliable way to verify the skills gained by learners with disabilities due to the absence of proper progress tracking and accessible certification.
4. Educators and mentors do not have a centralised platform to monitor learner progress, identify those who need extra support, and manage accessible learning materials effectively.

1.3 Objectives

This project aims to achieve the following objectives:

1. To analyze the accessibility requirements, learning challenges, and content delivery needs of learners with dyslexia, ADHD, and Autism Spectrum Disorder (ASD) through literature review and comparative study of existing e-learning platforms and assistive technology standards.
2. To develop a fully functional accessible web-based e-learning platform that incorporates rule-based adaptive content delivery, built-in Text-to-Speech (TTS) support, adjustable accessibility display settings, structured lesson and quiz management, and role-based user management for learners, educators, and administrators.
3. To implement a certificate generation module and an educator analytics dashboard that provide verifiable skill completion records and enable data-driven monitoring of individual learner progress.
4. To evaluate the functional correctness, usability, and accessibility of the developed platform through structured walkthroughs and expert review.

1.4 Scope

The scope of the ACES platform defines the boundaries of the system architecture, target users, and core functional modules. The system is designed to act as a supportive educational tool for both traditional and online learning environments.

1.4.1 Target Users

- **Learners:** The primary consumers of the educational content. The interface is specifically optimized for neurodivergent individuals, though it remains highly beneficial for neurotypical users.

- **Educators:** The course creators and administrators who require structured, intuitive tools to author accessible content, manage media assets, and monitor student progress.
- **Administrators:** System operators responsible for platform governance, user role management, and global accessibility configuration.

1.4.2 Core System Modules

The ACCESS architecture is divided into the following core modules:

- **Authentication and Profile Module:** Manages secure registration, login, role-based access control for learners, educators, and administrators, email verification, password reset, and profile management.
- **Accessibility and Personalization Engine:** The central differentiator of ACCESS. Stores per-user accessibility preferences defining cognitive profiles with granular controls over font, spacing, theme, focus mode, reduced motion, and text-to-speech. Settings are applied globally, requiring no manual configuration after onboarding.
- **Educator Authoring Module:** Provides an interface for creating structured courses with chapters and milestones, writing lessons, uploading video assets with time-coded questions, designing interactive activities, managing lesson version history, and configuring quiz, certificate, and achievement criteria.
- **Adaptive Learning Module:** Governs course enrollment, lesson progression, dynamic UI alterations based on accessibility profiles, and personalized recommendations triggered by quiz performance, including a rule-based engine that suggests revision, standard, or advanced content.
- **Interactive Assessment Module:** Handles timed and untimed quiz delivery, multiple attempts, automated grading, answer review, and pass/fail threshold evaluation that triggers remedial recommendations when scores fall below configured thresholds.

- **Analytics and Telemetry Module:** Tracks engagement metrics such as time per lesson, completion rates, and scores, and provides educators with per-student dashboards with at-risk detection, plus platform-wide analytics for administrators.
- **Certificate and Achievement Module:** Generates personalized PDF certificates upon course completion and administers a badge system that awards visual milestones based on predefined criteria to enhance motivation.
- **Interactive Content Module:** Supports multiple activity types with dedicated builder and viewer components, plus time-coded video questions and structured content blocks.
- **Communication and Notification Module:** Manages threaded lesson discussions, a contact form for inquiries, and a database-triggered notification system that alerts users on enrollment, progress, quiz attempts, lesson additions, and course publication.

1.5 Significance of Project

The development of the ACCESS platform represents a significant step towards educational equity. By shifting the burden of accessibility from the user to the system architecture, the platform ensures that neurodivergent learners can focus entirely on comprehending the educational material rather than struggling with the interface.

For learners, the project provides a calm, predictable, and highly customizable environment that mitigates anxiety and cognitive fatigue. For educators, it offers a streamlined toolset that enforces accessibility best practices during content creation, alongside deep analytical insights into student engagement. Ultimately, ACCESS demonstrates how modern web architecture can be leveraged to create a truly inclusive educational ecosystem applicable in both physical classrooms and remote learning scenarios.

1.6 Expected Output

The ACES platform is expected to produce the following deliverables upon completion:

- **Adaptive Web Application:** A fully functional, production-ready learning management system accessible via standard web browsers and capable of dynamically adjusting its interface based on user accessibility profiles.
- **Role-Based Dashboards:** Three distinct interfaces for Learners, Educators, and Administrators, each with tailored navigation, permissions, and tooling appropriate to their respective responsibilities.
- **Accessibility Customization System:** A comprehensive set of user-configurable accessibility presets covering cognitive profiles, including typography control, theme switching, focus mode, text-to-speech, and reduced motion toggles.
- **Adaptive Recommendation Engine:** A rule-based engine that automatically analyzes quiz performance and generates personalized lesson recommendations to address identified knowledge gaps.
- **Course Authoring Tools:** An editor integrated with media upload, interactive activity builders, and content embedding for educators.
- **Quiz and Assessment System:** A complete assessment lifecycle including timed and untimed quizzes, automated grading, multiple attempt management, and answer review capabilities.
- **Analytics Dashboard:** Visual engagement metrics and progress tracking with per-student and per-course views for educators, plus platform-wide analytics for administrators.
- **Certificate and Achievement System:** Automated PDF certificate generation upon course completion and a badge-based achievement system to incentivize learner progress.

- **Notification System:** Database-triggered real-time notifications for enrollment, quiz results, lesson additions, and course publication across all user roles.

1.7 Conclusion

This chapter established the foundational context for the ACCESS project by identifying the critical accessibility barriers present in current educational platforms, defining the specific problems to be addressed, and outlining the objectives and scope of the proposed system. The significance of the project was discussed in terms of its potential impact on educational equity for neurodivergent learners, and the expected deliverables were enumerated. The next chapter presents a comprehensive literature review, examining existing theoretical frameworks, analyzing current Learning Management Systems, describing the project methodology, and detailing the software and hardware requirements necessary to realize the ACCESS platform.

CHAPTER 2: LITERATURE REVIEW AND PROJECT METHODOLOGY

2.1 Introduction

The literature review provides the academic and technical foundation upon which the ACES platform is conceptualized. This chapter synthesizes contemporary research regarding cognitive accessibility in digital education, evaluates established theoretical frameworks such as Universal Design for Learning (UDL) [Meyer et al., 2014], and rigorously analyzes existing Learning Management Systems (LMS). By comparing platforms like Moodle [MoodleHQ, 2023b], Canvas [Instructure, Inc., 2023], Blackboard [Anthology Inc., 2023], and Google Classroom [Google LLC, 2023] against the specific requirements of neurodivergent learners, this chapter identifies the critical systemic gaps that justify the development of a natively adaptive, rule-based LMS architecture.

2.2 Theoretical Frameworks for Cognitive Accessibility

The design of digital interfaces for educational purposes must be grounded in empirically validated frameworks. The two primary pillars supporting the ACES architecture are the Universal Design for Learning (UDL) guidelines [Meyer et al., 2014] and the Web Content Accessibility Guidelines (WCAG) 2.2 [W3C Web Accessibility Initiative, 2023].

2.2.1 Universal Design for Learning (UDL)

Universal Design for Learning is a framework developed by CAST [Meyer et al., 2014] aimed at optimizing teaching and learning for all individuals based on scientific insights into how humans learn. UDL is predicated on three primary principles:

- **Multiple Means of Engagement:** Recognizing that learners differ markedly in the ways they are motivated. For neurodivergent learners, particularly those with ADHD, motivation can be severely hampered by unstructured environments [Barkley, 1997]. ACESS implements this by offering gamified achievements and structured, step-by-step progressive disclosure of course material to prevent cognitive overwhelm.
- **Multiple Means of Representation:** Learners vary in how they perceive and comprehend information. A Dyslexic learner may struggle with printed text but excel with audio. ACESS implements this principle via integrated Text-to-Speech (TTS) capabilities and dynamic CSS injection that allows the user to radically alter the typography (e.g., using Atkinson Hyperlegible [Atkinson, 2021]) and contrast ratios.
- **Multiple Means of Action and Expression:** Learners differ in how they navigate a learning environment and express what they know. ACESS supports this by providing multiple quiz formats (scenario-based, multiple-choice) and allowing untimed assessments to reduce anxiety-induced performance degradation.

2.2.2 Web Content Accessibility Guidelines (WCAG) 2.2

While UDL provides the pedagogical framework, WCAG 2.2 [W3C Web Accessibility Initiative, 2023] provides the strict, measurable technical parameters for digital accessibility. The ACESS platform targets Level AA compliance. Key WCAG criteria directly addressed by ACESS include:

- **Guideline 1.4.12 (Text Spacing):** The ability to override default text spacing (line height, letter spacing, word spacing) without loss of content or functionality. This is a critical requirement for Dyslexia, which ACESS solves via its Tailwind CSS `@theme` variable injection.
- **Guideline 2.2.1 (Timing Adjustable):** Providing users the ability to turn off, adjust, or extend time limits. ACESS allows educators to configure untimed quizzes specifically to accommodate learners requiring extended processing time.
- **Guideline 2.3.3 (Animation from Interactions):** Motion animation triggered by interaction can be disabled. The ACESS system implements a global attribute that globally disables all animations, critical for learners prone to vestibular disorders or severe distraction (ADHD).

2.3 Neurodivergent Profiles and Interface Interventions

Understanding the specific cognitive challenges faced by the target demographic is essential for architecting effective UI/UX interventions.

2.3.1 Attention Deficit Hyperactivity Disorder (ADHD)

ADHD is characterized by executive dysfunction [Zelazo et al., 2003], impacting working memory, task initiation, and sustained attention [Barkley, 1997]. In digital environments, learners with ADHD are highly susceptible to "saccadic fatigue," the exhaustion caused by the eye constantly darting across a busy screen to parse disorganized information [Munoz et al., 2003]. **ACESS Intervention:** The implementation of a strict "Focus Mode." This mode conditionally hides the primary navigation sidebar, the global footer, and all extraneous dashboard widgets when a learner enters a lesson. The content is restricted to a narrow, center-aligned column (maximum 65 characters per line), minimizing required eye movement and drastically reducing peripheral distraction. The technical implementation of Focus Mode, including

the activity diagram and database schema, is detailed in Chapter 4 (see Figures 4.2 and Tables 4.22, 4.23).

2.3.2 Dyslexia

Dyslexia involves difficulties with accurate or fluent word recognition and poor spelling and decoding abilities [Rello and Baeza-Yates, 2013]. Research indicates that specific typographical adjustments can significantly improve reading velocity and comprehension [Franceschini et al., 2013]. **ACCESS Intervention:** The platform abandons reliance on browser-level zoom. Instead, it utilizes React Context to allow the user to select a "Dyslexia Profile." This injects CSS variables that alter the global font family to specialized typefaces designed to prevent letter mirroring (e.g., confusing 'b' and 'd'), increases the base line-height to 1.75, and manipulates letter-spacing (tracking) to prevent visual crowding [Batanero et al., 2014]. The Dyslexia Profile implementation, including dynamic CSS injection and preference storage, is detailed in Chapter 4 (see Figures 4.1 and Tables 4.3, 4.2).

2.3.3 Autism Spectrum Disorder (ASD)

Learners on the Autism Spectrum often thrive in highly structured, predictable environments and can experience severe distress when confronted with unexpected auditory or visual stimuli [Lawson, 2011]. **ACCESS Intervention:** The platform enforces strict structural consistency. Educators utilizing the Tiptap rich-text editor [Brainhub, 2024] are prevented from creating chaotic, ad-hoc layouts. Furthermore, the system defaults to suppressing autoplaying media and allows the user to globally toggle a "Reduced Motion" state, ensuring the interface remains a calm, predictable schedule of events. The Reduced Motion and structural consistency features are implemented via the accessibility preference layer described in Chapter 4 (see Figures 4.3 and Tables 4.3).

2.4 Comparative Analysis of Existing LMS Platforms

To justify the development of ACES, an extensive evaluation of existing market-leading platforms was conducted. The platforms analyzed include Moodle [MoodleHQ, 2023b], Canvas LMS [Instructure, Inc., 2023], Blackboard Learn [Anthology Inc., 2023], and Google Classroom [Google LLC, 2023].

2.4.1 Moodle

Moodle is an open-source powerhouse, widely adopted globally [MoodleHQ, 2023a]. Its greatest strength is its modularity. However, this is also its critical failing regarding cognitive accessibility. **Limitations:** Moodle's native interface is notoriously dense and text-heavy. While administrators can install accessibility plugins like the 'Accessibility block,' these are bolt-on solutions. They require the learner to actively engage with a separate menu to adjust settings, adding cognitive load. Furthermore, Moodle relies on educators to manually identify struggling students and assign remedial work. It lacks a native, real-time heuristic engine to automate this process.

2.4.2 Canvas LMS

Canvas by Instructure represents a more modern, streamlined UI compared to Moodle [Instructure, Inc., 2023]. It utilizes a card-based dashboard that is visually appealing to neurotypical users. **Limitations:** Canvas achieves basic WCAG compliance but fails to provide deep cognitive accommodations. It lacks native Text-to-Speech, forcing reliance on OS-level tools, and does not offer granular control over typography such as specific Dyslexia fonts or adjusted inter-letter spacing. The platform is highly linear. If a student fails a Canvas quiz, the system cannot automatically restructure the module path to enforce revision.

2.4.3 Blackboard Learn

Blackboard is an enterprise-grade solution focused heavily on synchronous learning integrations and comprehensive grading rubrics [Anthology Inc., 2023].

Limitations: Blackboard’s interface is often criticized for complex, deeply nested navigation menus (often three or four clicks deep to access a specific assignment file). For learners with executive dysfunction (ADHD), this navigational friction is a massive barrier to task initiation. Like Moodle, Blackboard’s accessibility features are largely reactive (e.g., Blackboard ALLY checks documents for screen-reader compatibility) rather than proactive (dynamically altering the interface for the user).

2.4.4 Google Classroom

Google Classroom is a stripped-down, lightweight system focused on integration with the Google Workspace ecosystem (Docs, Drive, Forms) [Google LLC, 2023].

Limitations: While its minimalist design reduces initial visual clutter, it is too rudimentary to function as a fully adaptive LMS. It completely lacks advanced features such as rule-based learning paths, granular progress telemetry (tracking time-spent-per-lesson), or integrated cognitive profiles. It functions more as an assignment drop-box than a pedagogical support engine.

2.5 Comparison Matrix

Table 2.1 provides a structured summary comparing the features of the analyzed platforms against the proposed ACCESS system.

Each entry in Table 2.1 is derived from the platform-specific analyses presented in Section 2.4.1, with qualitative assessments (“Partial”, “No”) supported by the cited documentation and literature. For example, Moodle’s “Partial” WCAG compliance reflects its reliance on bolt-on accessibility plugins rather than native enforcement [MoodleHQ, 2023b], while Canvas’s lack of native TTS is documented in its

Table 2.1: LMS Feature Comparison Matrix

Feature Domain	Moodle	Canvas	Blackboard	Google Classroom	Proposed (ACCESS)
Native TTS	Plugin Req.	No	No	OS Reliant	Yes (Built-in)
Dynamic Fonts (Dyslexia)	Theme Req.	No	No	No	Yes (CSS Vars)
Automated Remedial Paths	No	No	No	No	Yes (Rule-based)
Focus Mode (ADHD)	No	No	No	Partial	Yes (Strict Route)
Gamification/Achievements	Plugin Req.	External App	No	No	Yes (Native)
WCAG 2.2 Level AA	Partial	Yes	Yes	Yes	Yes (Enforced)
Granular Analytics	Yes	Yes	Yes	No	Yes (Recharts)
Reduced Motion Toggle	No	No	No	No	Yes (Global State)

accessibility guides [Instructure, Inc., 2023].

2.5.1 Alternative Approaches and Justification

Several alternative technical approaches were evaluated during the design of the ACCESS platform before arriving at the selected architecture:

- **Monolithic Architecture vs. Decoupled Client-Server:** A monolithic Ruby on Rails or PHP Laravel application was considered for its simpler deployment model. However, this approach was rejected in favor of a decoupled Next.js frontend [Vercel Inc., 2023a] with Supabase backend-as-a-service [Supabase, 2023], which allows the accessibility engine to dynamically inject CSS variables on the client without full page reloads, preserving the fluid user experience critical for ADHD learners.

- **CSS-in-JS Libraries vs. Tailwind CSS:** Alternatives such as styled-components and Emotion were evaluated. While they offer dynamic styling, they introduce runtime overhead that degrades performance on low-end devices. Tailwind CSS v4 [Tailwind Labs, 2023] with Just-in-Time compilation and CSS custom properties was selected because it generates minimal production CSS and enables instantaneous theme switching without JavaScript re-computation.
- **Document Database vs. Relational Database:** MongoDB was considered for its schema flexibility. However, the relational integrity enforced by PostgreSQL with foreign key constraints and Row-Level Security [Supabase, 2024] was deemed essential for ensuring data consistency across enrollments, progress tracking, and quiz attempts, particularly given the complex relational model required by the adaptive recommendation engine.
- **Traditional State Management vs. React Context:** Redux was considered for global state management but was deemed overly verbose for the accessibility settings use case. React Context with the `AccessibilityProvider` pattern was chosen for its simplicity [Meta Platforms, Inc., 2023], native React integration, and minimal boilerplate when propagating CSS theme attributes across the component tree.

2.6 Project Methodology

The ACCESS platform was developed using the **Agile Scrum** methodology [Schwaber, 1997], an iterative framework that emphasizes continuous delivery, regular stakeholder feedback, and adaptive planning. Scrum was selected over traditional sequential models such as Waterfall because designing for cognitive accessibility is an inherently empirical process. Assumptions about the effectiveness of specific interface interventions, such as a Dyslexia font or Focus Mode layout, must be validated through iterative user testing rather than deferred to the final stages of development.

The development lifecycle was organized into six sprints, each spanning two weeks:

- **Sprint 0: Requirements and Scaffolding.** Definition of the product backlog based on WCAG 2.2 guidelines and UDL principles. Establishment of the Next.js project structure, Supabase project initialization, and PostgreSQL schema design.
- **Sprints 1-2: Core Architecture.** Implementation of user authentication with Supabase Auth, database schema normalization with Row-Level Security policies, and basic role-based routing via middleware.
- **Sprints 3-4: Feature Development.** Development of the Educator authoring interface (Tiptap editor, course creation, quiz builder) and the Learner experience (course enrollment, lesson viewing, progress tracking).
- **Sprints 5-6: Advanced Features and Refinement.** Integration of the rule-based adaptive recommendation engine, accessibility personalization engine with CSS variable injection, analytics dashboards, certificate generation, and notification triggers.

Each sprint concluded with a review and retrospective, ensuring continuous alignment with project objectives. A detailed breakdown of the Agile implementation is provided in Chapter 3.

2.7 Project Requirements

2.7.1 Software Requirements

The following software tools and technologies were utilized in the development and deployment of the ACES platform:

- **Visual Studio Code:** Primary integrated development environment for writing and debugging code, with extensions for TypeScript, Tailwind CSS, and ESLint integration.
- **Node.js:** JavaScript runtime environment required to execute the Next.js development server and build toolchain.

- **Git and GitHub:** Version control system for source code management and collaboration, with the repository hosted on GitHub for continuous integration with Vercel.
- **Supabase CLI:** Command-line tool for local database migrations, schema management, and TypeScript type generation for the PostgreSQL backend.
- **Vercel [Vercel Inc., 2023b]:** Cloud platform for deploying the Next.js application, providing serverless edge functions, automatic HTTPS, and continuous deployment from the GitHub repository.
- **Next.js 16 [Vercel Inc., 2023a]:** React framework with App Router providing server-side rendering, static generation, and middleware-based route protection.
- **TypeScript [Microsoft, 2023]:** Statically typed superset of JavaScript used throughout the codebase for improved code quality and maintainability.
- **Tailwind CSS v4 [Tailwind Labs, 2023]:** Utility-first CSS framework with Just-in-Time compilation, enabling dynamic theme injection via CSS custom properties for accessibility customization.
- **Supabase (PostgreSQL) [Supabase, 2023]:** Backend-as-a-service providing a managed PostgreSQL database with Row-Level Security, authentication, and file storage capabilities.
- **Framer Motion [Framer, 2024]:** Animation library for React used to implement smooth transitions while respecting the user's reduced-motion accessibility preference.
- **Tiptap [Brainhub, 2024]:** Headless rich-text editor built on ProseMirror, integrated into the Educator authoring workflow for creating structured lesson content.
- **Recharts [Recharts Team, 2024]:** Charting library for React used to render engagement analytics and progress visualizations on educator and administrator dashboards.

- **Figma [Figma, 2023]:** UI/UX design tool used to prototype interface layouts and accessibility component mockups prior to implementation.

2.7.2 Hardware Requirements

The following hardware specifications were used during development and are recommended for optimal deployment:

- **Development Workstation:** A personal computer with a minimum of 8 GB RAM, a multi-core processor (Intel Core i5 or equivalent), and a solid-state drive with at least 256 GB of storage for running the development server, database migrations, and local testing.
- **Display:** A monitor with a minimum resolution of 1920x1080 pixels to adequately preview responsive layouts and accessibility adjustments across viewport sizes.
- **Network Connectivity:** A stable broadband internet connection with at least 10 Mbps download speed for accessing cloud services (Supabase, Vercel, GitHub).
- **Server Infrastructure:** Cloud-based infrastructure provided by Vercel (edge functions and static hosting) and Supabase (managed PostgreSQL and file storage), eliminating the need for dedicated physical server hardware.

2.7.3 Other Requirements

- **Supabase Account:** A cloud-hosted Supabase project for database management, authentication configuration, and storage bucket policies.
- **GitHub Account:** Repository hosting for source code version control and integration with Vercel for continuous deployment.
- **Vercel Account:** Deployment platform account linked to the GitHub repository for automated builds and hosting.

- **SMTP Email Service:** A Gmail SMTP configuration for sending password reset emails and account verification notifications via Nodemailer.
- **Web Browser:** A modern browser (Google Chrome, Mozilla Firefox, or Microsoft Edge) for testing and accessing the deployed application.

2.8 Project Schedule and Milestones

The ACESS project was executed across six Agile Scrum sprints, each spanning two weeks, for a total development duration of twelve weeks. Table 2.2 summarizes the milestone schedule.

Table 2.2: Project Schedule and Milestones

Sprint	Focus Area	Key Deliverables
0	Requirements and Scaffolding	Product backlog, Next.js project setup, Supabase initialization, database schema design
1	Core Authentication	User registration and login, role-based middleware, email verification flow
2	Database and Security	Row-Level Security policies, storage bucket configuration, data dictionary implementation
3	Educator Authoring	Course creation interface, Tiptap editor integration, quiz builder, media upload
4	Learner Experience	Course enrollment, lesson viewer, progress tracking, Focus Mode implementation
5	Adaptive and Analytics	Recommendation engine, accessibility presets, analytics dashboards, certificates, notifications

The Gantt chart in Figure 2.1 provides a visual representation of the project timeline, illustrating the scheduling and duration of each development phase across 14 weeks.

Phase	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Requirements Gathering														
System Design														
Core Architecture														
Feature Development														
Advanced Features														
Testing & Refinement														
Documentation & Deployment														

Figure 2.1: Gantt chart of ACCESS development schedule (14 weeks)

2.9 Summary

The literature review demonstrates that while current LMS platforms provide robust administrative and content-hosting capabilities, they systematically fail to address the specific cognitive requirements of neurodivergent learners. Reliance on external plugins, static content delivery models, and dense navigation hierarchies induce significant extraneous cognitive load [Sweller, 1988]. The theoretical frameworks of UDL and WCAG 2.2 demand a more integrated approach. The comparative analysis definitively justifies the engineering of the ACCESS platform: a system built from the database schema upward to be natively adaptive, utilizing dynamic CSS injection and rule-based server logic to provide an equitable, highly personalized learning environment. The project methodology, requirements, and schedule outlined in this chapter establish the roadmap for implementing the ACCESS system in the subsequent phases.

CHAPTER 3: SYSTEM ANALYSIS

3.1 Introduction

This chapter transitions the theoretical justifications established in the literature review into concrete software engineering specifications. It begins with a detailed problem analysis of existing educational platforms, followed by a comprehensive requirements analysis covering data, functional, non-functional, and other requirements. The chapter then presents the system architecture, module design, and deployment considerations necessary to execute a secure, highly accessible Learning Management System.

3.2 Problem Analysis

The ACESS platform is designed to address specific, systemic shortcomings in current educational technology. While the literature review in Chapter 2 established the general limitations of platforms such as Moodle, Canvas, Blackboard, and Google Classroom, this section analyzes the concrete operational problems that directly influenced the architectural decisions of ACESS.

3.2.1 Current System Scenario

Existing Learning Management Systems typically operate under a one-size-fits-all paradigm. Learners are presented with a fixed interface regardless of their cognitive profile, forcing neurodivergent users to seek third-party accommodations such as

browser extensions for text-to-speech, external tools for font customization, or manual workarounds for managing visual clutter. This places the burden of accessibility on the learner rather than the system.

From a data flow perspective, current platforms process learner interactions in a linear fashion: enrollment, content delivery, assessment, and completion. They lack the feedback loops necessary to detect when a learner is struggling and respond adaptively. For example, if a student performs poorly on a quiz about database normalization, the system continues to present the next topic as though no intervention is needed.

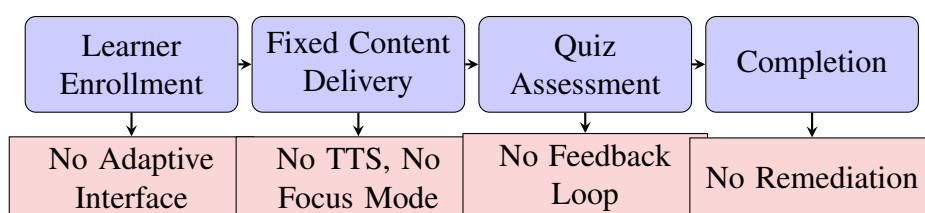


Figure 3.1: Current System Problem Analysis Diagram

3.2.2 Detailed Problem Breakdown

The problems identified in Chapter 1 are analyzed here in greater technical depth:

1. **Rigid Typography and Layout:** Standard LMS platforms use fixed CSS stylesheets that do not respond to individual user needs. Learners with Dyslexia require specific typographic conditions, including increased letter spacing (tracking) of at least 0.12em, line height of 1.5 to 1.8, and sans-serif fonts such as Atkinson Hyperlegible [Atkinson, 2021] designed to prevent letter confusion, such as differentiating 'b' from 'd' [Batanero et al., 2014]. Without these adjustments, reading velocity decreases and cognitive fatigue accelerates, leading to early lesson abandonment.
2. **Cognitive Overload from Navigation:** Platforms like Moodle and Blackboard present deeply nested navigation menus (often three to four levels deep) combined with dense dashboard widgets showing calendars, announcements, grades, and

messages simultaneously. For learners with ADHD [Barkley, 1997], this multi-stimulus environment triggers rapid attention fragmentation, making it difficult to sustain focus on a single learning objective. The absence of a distraction-free mode means learners must manually close or minimize interface elements.

3. **Lack of Adaptive Remediation:** Current assessment systems grade quizzes and record scores but do not analyze performance patterns to generate corrective action. When a learner fails a prerequisite concept, the system does not automatically unlock remedial content or recommend revision. This linear progression model assumes uniform learner readiness, which is particularly detrimental for neurodivergent students who may require repeated exposure to foundational topics before advancing.

3.3 Requirement Analysis

3.3.1 Data Requirement

The ACCESS platform requires a robust relational data model to manage the complex relationships between users, courses, lessons, assessments, and accessibility preferences. The system must store and process the following categories of data:

- **User Data:** Authentication credentials (email, password hash), profile information (name, avatar, role), and notification preferences. User data must be isolated by role with strict access controls.
- **Accessibility Data:** Per-user cognitive profile settings including disability type, preferred font family and size, theme (light, dark, high-contrast, soft), line spacing, text-to-speech and caption toggles, reduced motion preference, simplified UI mode, and keyboard navigation flags. These settings must persist across sessions and apply globally to all interface components.

- **Curriculum Data:** Hierarchical course structures (courses, chapters, lessons) with rich-text HTML content, video assets, transcripts, and metadata such as difficulty level, estimated duration, and prerequisite relationships that drive the adaptive engine.
- **Assessment Data:** Quiz configurations (time limits, pass thresholds, max attempts), question banks with multiple-choice and scenario formats, answer options with correctness flags, and learner attempt records including scores and answer selections.
- **Telemetry Data:** Granular engagement metrics including lesson view status, time spent per lesson (in seconds), enrollment status (active, completed, dropped), and quiz attempt history with pass/fail outcomes.
- **Content and Achievement Data:** Interactive activity configurations (flashcards, drag-and-drop, memory games, timelines), certificate records with unique reference codes, and badge/achievement definitions and user progress toward them.
- **Notification and Communication Data:** System-generated notifications triggered by enrollment, quiz attempts, lesson additions, and course publications, along with lesson-level threaded comments and contact form submissions.

The detailed database schema and data dictionary for these entities are presented in Chapter 4, Section 4.3.

3.3.2 Functional Requirement

Functional requirements define the explicit behaviors, processes, and services the system must execute. They are derived directly from the system's operational architecture and are categorized by user role.

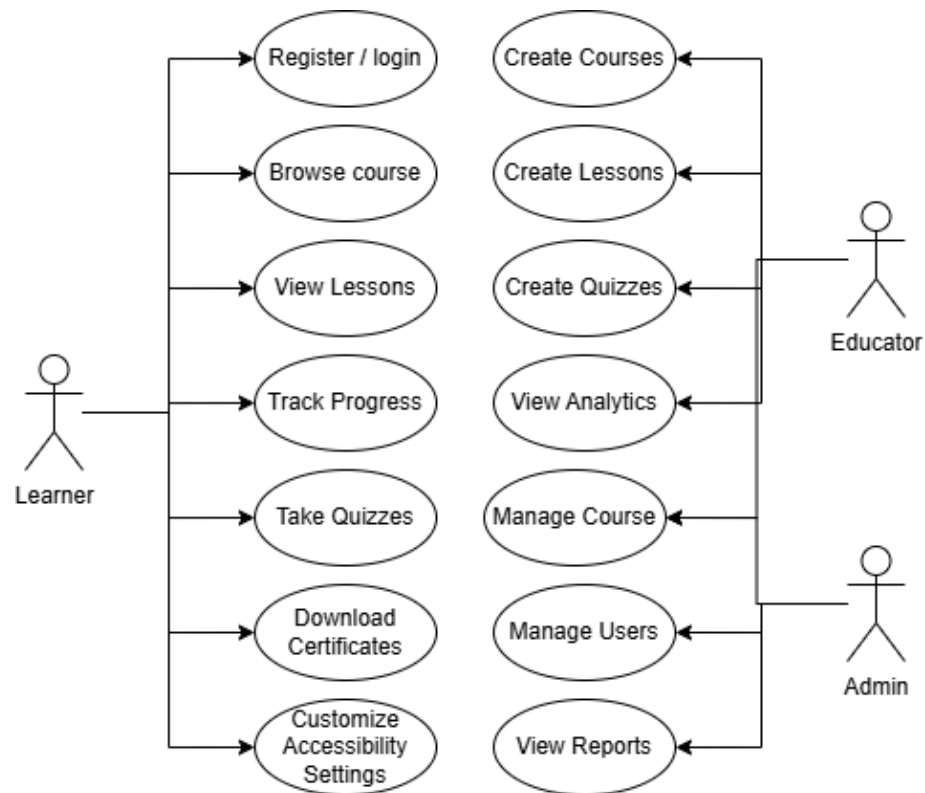


Figure 3.2: Use Case Diagram for ACCESS Platform

Use Case Diagram Figure 3.2 visually maps the interactions between the primary actors (Student, Educator, Admin) and the system's functional boundaries, which are explicitly detailed in the subsequent tables.

Table 3.1: Functional Requirements: Student Module

Req ID	Description	Actor	Inputs	Outputs
FR-STU-01	Registration and secure login.	Student	Email, Password	Authenticated JWT Session.
FR-STU-02	Accessibility profile selection.	Student	Theme, Font, TTS preferences	Updated user_profiles.accessibility_prefs record; UI dynamically alters.
FR-STU-03	Course discovery and enrollment.	Student	course_id selection	INSERT into enrollments; unlocks course content.
FR-STU-04	Lesson viewing with Focus Mode.	Student	Navigation to '/lesson/[id]'	Renders sanitized HTML; hides global navigation elements.
FR-STU-05	Interactive video questions.	Student	Video timestamp trigger	Pauses video, renders embedded question overlay.
FR-STU-06	Quiz participation.	Student	Selected quiz_options	Evaluates score; INSERTs into quiz_attempts.
FR-STU-07	Automated Progress tracking.	Student	Dwell time on lesson route	UPDATEs time_spent in lesson_progress.
FR-STU-08	Achievement earning.	Student	100% module completion	Unlocks gamified badge in UI.
FR-STU-09	Certificate generation/viewing.	Student	100% course completion	Generates verifiable PDF; displays download link.

Student Functions

Table 3.2: Functional Requirements: Educator Module

Req ID	Description	Actor	Inputs	Outputs
FR-EDU-01	Course creation and editing.	Educator	Title, metadata, difficulty_level	INSERT/UPDATE courses table.
FR-EDU-02	Lesson creation via Tiptap.	Educator	Rich-text content	Sanitized HTML string saved to lessons.content_html.
FR-EDU-03	Quiz and Activity creation.	Educator	Question text, Correct option flag	Populates quiz_questions and quiz_options linked via FK.
FR-EDU-04	Asset management.	Educator	MP4/Image files	Uploads to Supabase Storage; returns secure URL.
FR-EDU-05	Student monitoring & Analytics.	Educator	Selection of course_id	Renders Recharts graphs aggregating lesson_progress data.
FR-EDU-06	Achievement configuration.	Educator	Threshold values	Maps achievements to specific course_milestones.

Educator Functions

Table 3.3: Functional Requirements: Admin Module

Req ID	Description	Actor	Inputs	Outputs
FR-ADM-01	User role management.	Admin	Target user_id, new role string	UPDATEs users.role allowing bypass of middleware RBAC.
FR-ADM-02	Course moderation pipeline.	Admin	Review action (Approve/Reject)	UPDATEs courses.status from pending_review to published.
FR-ADM-03	Accessibility preset management.	Admin	Global CSS variable thresholds	Updates system default fallbacks for Dyslexia/ADHD themes.
FR-ADM-04	System course creation.	Admin	Course metadata	Flags course as system_course=true, bypassing educator limits.
FR-ADM-05	Global Analytics monitoring.	Admin	Date range filters	Aggregates platform-wide engagement metrics.

Admin Functions

3.3.3 Non-Functional Requirement

Non-functional requirements (NFRs) specify the critical quality attributes and systemic constraints that govern the operation of the ACESS platform.

Performance

- **Page Response Time:** Next.js Client-side route transitions must execute within **300ms** to prevent cognitive disruption and maintain user focus.
- **Course Loading:** Server-side data fetching for initial course loads must resolve within **800ms**, leveraging edge caching where applicable.
- **Video Loading:** Embedded instructional videos must commence playback within **2 seconds** utilizing adaptive bitrate streaming via Supabase Storage CDNs.
- **Concurrent Users:** The PostgreSQL database, utilizing connection pooling (PgBouncer), must support a minimum of **500 concurrent** intensive operations (e.g., simultaneous quiz submissions) without degrading frontend TTFB (Time to First Byte).

Security

- **Authentication:** All user sessions must be cryptographically secured using JSON Web Tokens (JWT) managed by Supabase Auth, with a mandatory 15-minute inactivity timeout.
- **Authorization (RBAC):** The application middleware must definitively intercept and route requests based on the role claim within the JWT, achieving zero-trust client routing.

- **Database Protection:** Strict Row-Level Security (RLS) must be enabled on 100% of tables containing Personally Identifiable Information (PII) or cognitive telemetry, ensuring isolation at the storage layer.
- **Input Validation:** All API endpoints must utilize Zod schema validation to sanitize incoming payloads, preventing SQL injection and Cross-Site Scripting (XSS).

Accessibility

- **WCAG Compliance:** The platform must strictly adhere to **WCAG 2.2 Level AA** standards [W3C Web Accessibility Initiative, 2023]. Minimum contrast ratios must exceed 4.5:1 for standard text.
- **Dyslexia Support:** The system must natively provide the Atkinson Hyperlegible typeface [Atkinson, 2021] and allow users to dynamically increase line-spacing to 1.75em.
- **ADHD and Autism Support:** The implementation of a global reduced-motion attribute must programmatically disable all CSS and JavaScript animations [Framer, 2024]. Focus Mode must isolate content to a 65-character maximum width to reduce saccadic fatigue.
- **Screen Reader Compatibility:** All custom UI components must natively handle WAI-ARIA states such as aria-expanded and aria-hidden, and support complete keyboard navigation.

Usability

- **Consistent Navigation:** The primary navigational architecture must remain structurally identical across all modules to ensure predictability for Autistic learners.
- **Error Prevention:** Destructive actions (e.g., an Educator deleting a course) must require a confirmed multi-step verification modal to prevent accidental data loss.

Reliability

- **Data Integrity:** The database schema must strictly enforce foreign key constraints with cascade deletes where appropriate to prevent orphaned records, such as a lesson progress record existing without a valid enrollment.
- **Backup Strategy:** The Supabase infrastructure must be configured to execute automated, daily Point-in-Time Recovery (PITR) backups of the entire PostgreSQL volume.

Maintainability and Scalability

- **Modular Architecture:** The codebase must adhere to strict separation of concerns, decoupling the data-fetching logic (Server Components) from the interactive accessibility logic (Client Components).
- **Database Normalization:** The schema must adhere to the Third Normal Form (3NF) to eliminate data redundancy, ensuring the database scales efficiently as course and user volume increases.

3.3.4 Others Requirement

The software and hardware tools selected for the ACES platform, as listed in Chapter 2, Section 2.4, are justified by the following architectural considerations:

- **Next.js 16 and React [Vercel Inc., 2023a]:** Server-side rendering and React Server Components reduce initial JavaScript payload, critical for rapid page loads that prevent cognitive drop-off for ADHD learners. The App Router provides middleware-based route protection that enforces role-based access at the edge.
- **Supabase with PostgreSQL [Supabase, 2023]:** Row-Level Security provides data isolation [Supabase, 2024] at the database level rather than relying solely on

application-layer guards, ensuring that cognitive telemetry and assessment data remain cryptographically isolated per user.

- **Tailwind CSS v4 [Tailwind Labs, 2023]:** Just-in-Time compilation enables dynamic CSS variable injection, allowing instantaneous theme and typography switching without JavaScript re-evaluation, which is essential for real-time accessibility adjustments.
- **TypeScript [Microsoft, 2023]:** Static typing across the full stack reduces runtime errors in critical paths such as quiz scoring and accessibility profile application, where a type mismatch could result in incorrect interface rendering.
- **Vercel [Vercel Inc., 2023b]:** Serverless edge deployment provides global low-latency access to the platform, which is particularly important for learners whose cognitive focus may be disrupted by slow page transitions.

3.4 System Architecture

The ACESS platform employs a modern, decoupled client-server architecture, leveraging edge computing paradigms to maximize performance and security.

3.4.1 Context Diagram

The Context Diagram establishes the highest-level view of the system, illustrating the boundaries of the ACESS platform and its interactions with external entities.

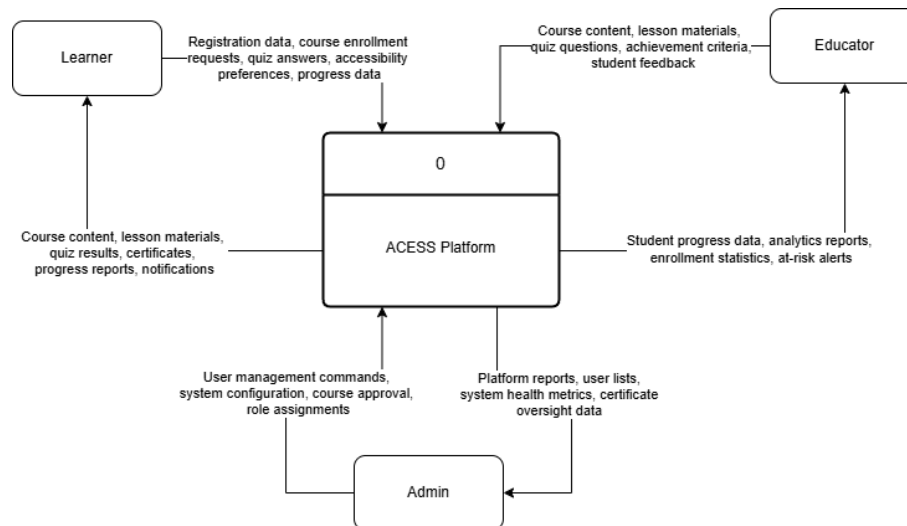


Figure 3.3: Context Diagram

As depicted in Figure 3.3, the ACCESS system acts as the central processing hub. The primary external entities are the Learner (who consumes content and provides telemetry), the Educator (who authors content and configures assessments), and the Administrator (who governs system integrity). The system also interfaces with external secure email providers (for authentication/password resets) and potentially external text-to-speech APIs, though native browser synthesis is prioritized.

3.4.2 System Architecture Diagram

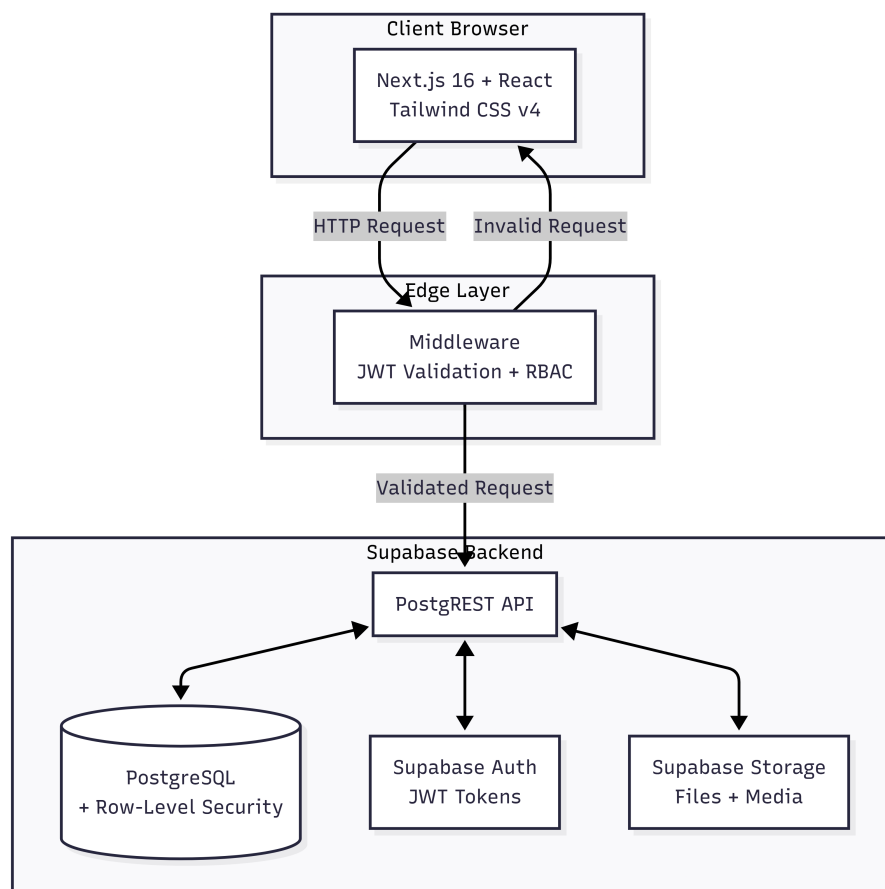


Figure 3.4: System Architecture Diagram

Figure 3.4 details the three-tier decoupled architecture:

- **Frontend Architecture (Next.js 16 and React):** The presentation layer is built on the Next.js App Router. It utilizes React Server Components to execute data fetching securely on the server, drastically reducing the JavaScript payload delivered to the client. This is crucial for rapid page loads, preventing cognitive drop-off for ADHD learners. Client components are restricted to highly interactive elements, such as the rich-text editor and interactive quiz interfaces. The UI is styled utilizing Tailwind CSS v4, which enables the dynamic injection of CSS variables for accessibility toggling.
- **Middleware and Authentication Layer:** A Next.js Edge Middleware intercepts every incoming request. It interfaces with the authentication system to validate

JWT cookies and extract the user's role. The middleware enforces strict Role-Based Access Control (RBAC), instantly redirecting unauthorized attempts before the page is even rendered.

- **Backend and Database Architecture (Supabase):** Supabase acts as the comprehensive Backend-as-a-Service. It provides a managed PostgreSQL database, handling complex relational queries. Crucially, data security is not solely reliant on the API layer. PostgreSQL Row-Level Security policies are inscribed directly into the database schema, ensuring users can only read, create, or update records they own.
- **Storage Architecture:** Supabase Storage buckets manage unstructured assets such as course thumbnails, PDF certificates, and embedded videos. These buckets are governed by parallel security policies, ensuring private assets remain inaccessible to unauthenticated requests.

3.4.3 Deployment Diagram

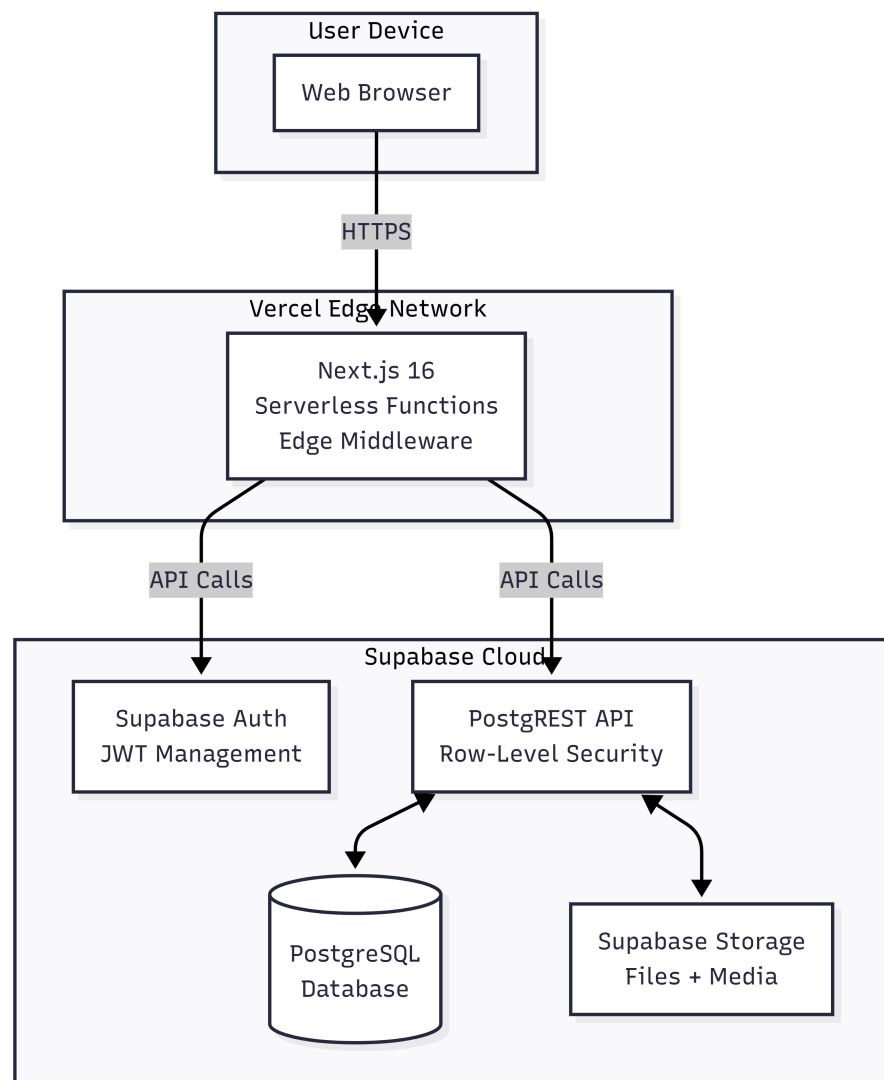


Figure 3.5: Deployment Diagram

The Deployment Diagram (Figure 3.5) illustrates the physical distribution of the system components. The Next.js frontend is deployed to a serverless edge network (e.g., Vercel or AWS Amplify), ensuring high availability and low latency globally. The Supabase backend operates on dedicated cloud infrastructure, exposing a secure REST/GraphQL API via PostgREST, which the Next.js server components consume securely.

3.5 Module Design

The system is logically partitioned into discrete, highly cohesive modules to facilitate maintainability.

- **Student Module:** Encapsulates all logic related to course discovery, enrollment, lesson consumption, and quiz execution.
- **Educator Module:** Manages the creation pipeline. Handles the rich-text editor state, quiz generation, and the uploading of media assets to storage.
- **Admin Module:** Provides a high-level oversight interface for user management, role reassignment, and global accessibility preset configurations.
- **Accessibility Module:** A globally available React Context Provider that subscribes to the user's preferences and dynamically manipulates the DOM root attributes for theme and motion settings.
- **Interactive Activity Module:** Governs the execution of varied assessment types, including standard multiple-choice quizzes and embedded video questions, immediately evaluating responses in the backend.
- **Analytics Module:** Aggregates data from lesson progress and quiz attempts to render visualizations, providing educators with actionable intelligence on cohort performance.

3.6 Summary

This chapter analyzed the engineering specifications required to build the ACES platform. It began with a detailed problem analysis of existing educational systems, followed by a comprehensive requirements analysis covering data, functional, non-functional, and other requirements. The system architecture was presented through Context, Architecture, and Deployment diagrams, and the modular decomposition of the

system was described. The subsequent chapter will translate these requirements into concrete database schemas and interface designs.

CHAPTER 4: SYSTEM DESIGN

4.1 Introduction

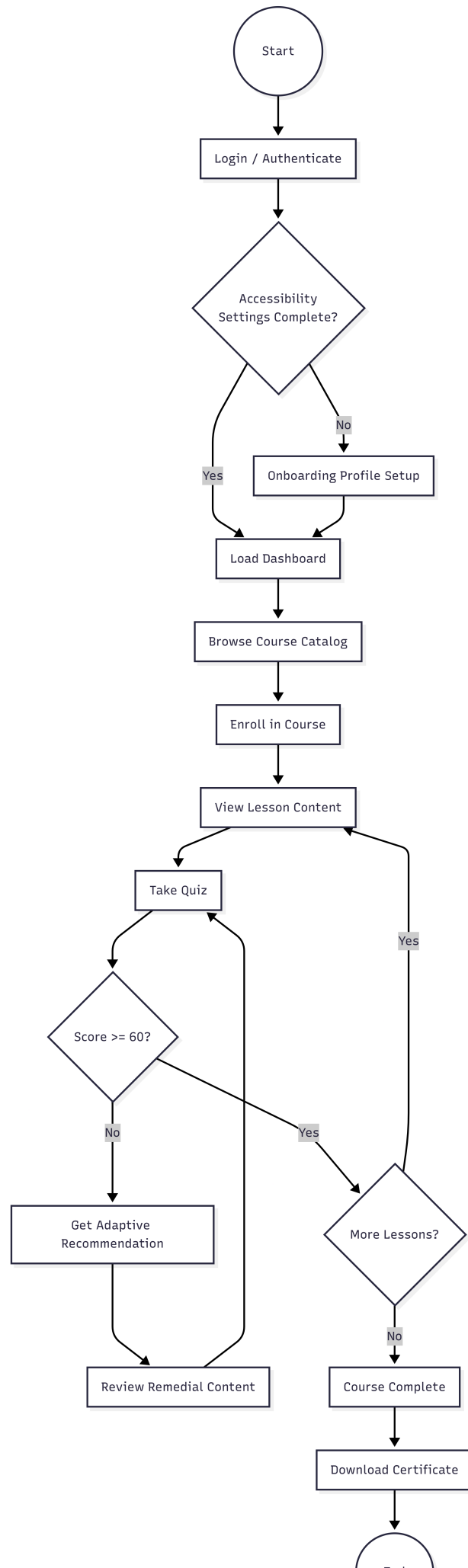
This chapter translates the conceptual requirements established in the previous phase into concrete, highly granular engineering designs. It is structured according to the High-Level Design and Detailed Design phases. High-Level Design encompasses behavioral and interaction diagrams, user interface design specifications, and the conceptual database model. Detailed Design covers the software component architecture, physical database implementation, and systemic implementation details of the ACCESS platform.

4.2 High-Level Design

4.2.1 Behavioral Design (Activity Diagrams)

Activity diagrams model the workflow of the system, illustrating the procedural logic executed by the different actors.

Student Activity Diagram



As shown in Figure 4.1, the Student workflow is highly structured. Upon authentication, the system evaluates the `user_accessibility_settings`. If incomplete, the user is routed to an onboarding profile setup. Once the dashboard loads, the core loop involves browsing the catalog, initializing an enrollment, and sequentially consuming lessons. The critical branching logic occurs post-assessment: if a quiz score is below the predefined threshold, the system diverges from the linear path, querying the `recommendations` table and forcing the student to review foundational material before progressing.

Educator Activity Diagram

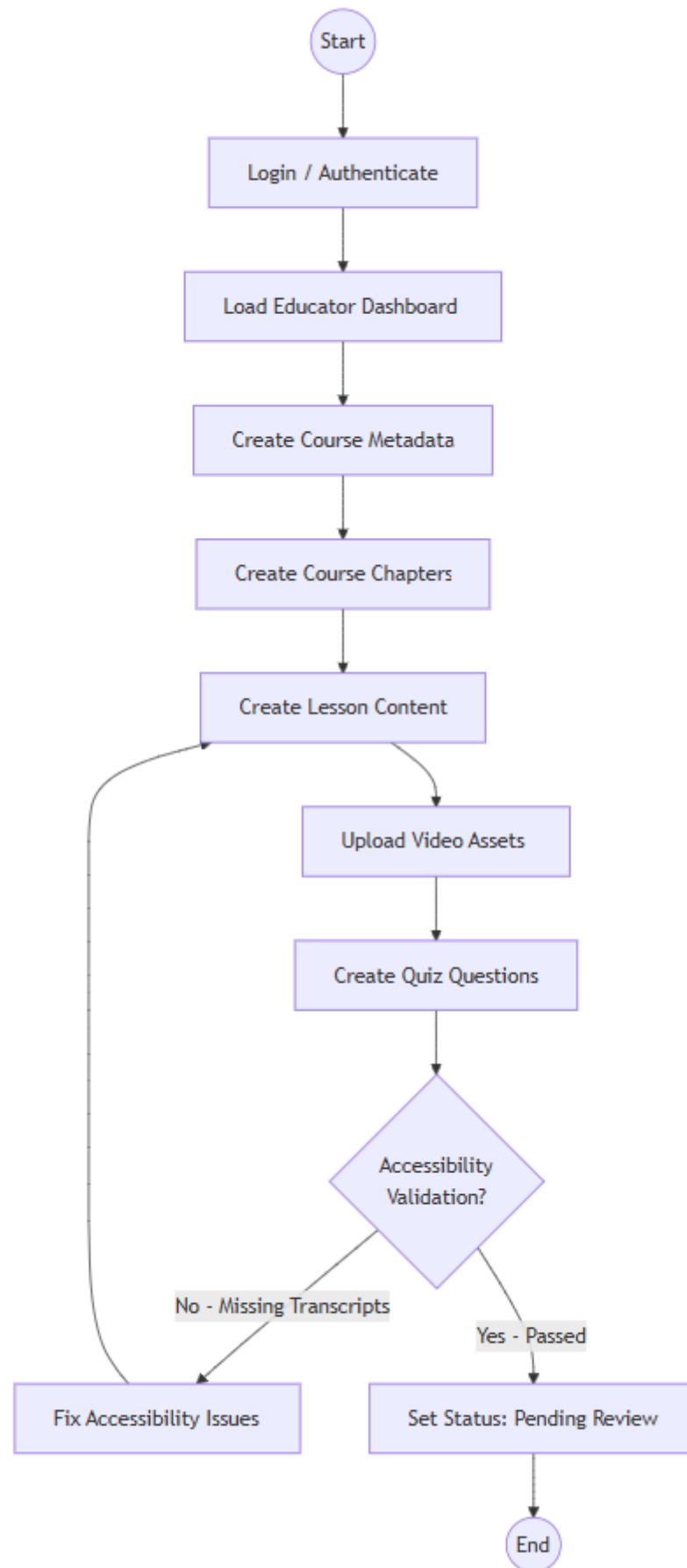


Figure 4.2: Educator Activity Diagram

Figure 4.2 maps the Educator's authoring workflow. The process begins with establishing course metadata and structure (`course_chapters`). The educator then enters a loop of content creation: drafting lessons via Tiptap, uploading video assets, and defining `quiz_questions`. A critical validation node exists before publishing: the system checks for mandatory accessibility flags (e.g., verifying video transcripts are attached). Upon passing validation, the course status updates to `pending_review`.

Administrator Activity Diagram

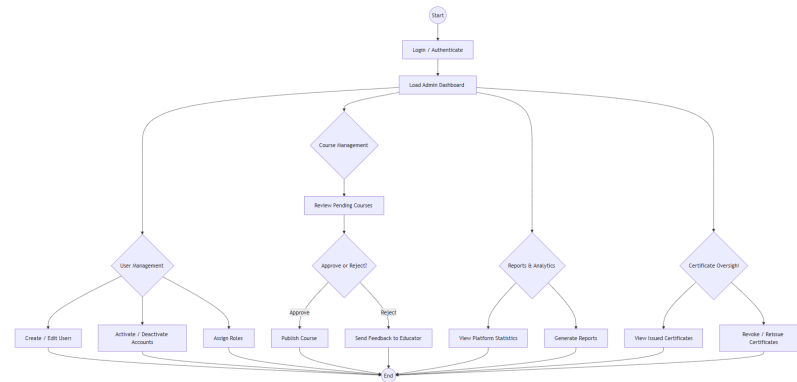


Figure 4.3: Administrator Activity Diagram

The Administrator workflow (Figure 4.3) is primarily reactive. The admin monitors the moderation queue. When a course enters `pending_review`, the admin evaluates it against overarching accessibility guidelines. If rejected, a notification payload is dispatched to the educator. If approved, the course is published. Furthermore, the admin workflow includes auditing user logs and modifying global system variables.

4.2.2 Interaction Design (Sequence Diagrams)

Sequence diagrams detail how objects interact over time to execute specific Use Cases, mapping the exact API calls between the Next.js client, the Edge Middleware, and the Supabase backend.

Sequence: Course Enrollment

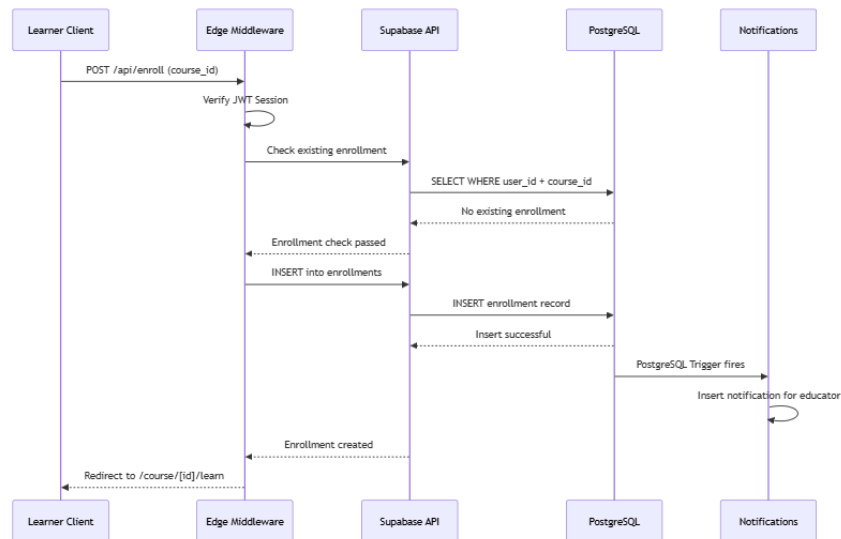


Figure 4.4: Course Enrollment Sequence Diagram

Figure 4.4 illustrates the enrollment transaction. 1. The Learner Client sends a POST `/api/enroll` request containing the `course_id`. 2. The Edge Middleware intercepts, verifying the JWT session. 3. The API queries Supabase to check for existing enrollments (preventing duplicates). 4. An 'INSERT' operation is executed on the 'enrollments' table. 5. A PostgreSQL Trigger fires, executing a webhook that inserts a record into the 'notifications' table, alerting the educator. 6. The client is redirected to the `/course/[id]/learn` route.

Sequence: Lesson Learning & Telemetry

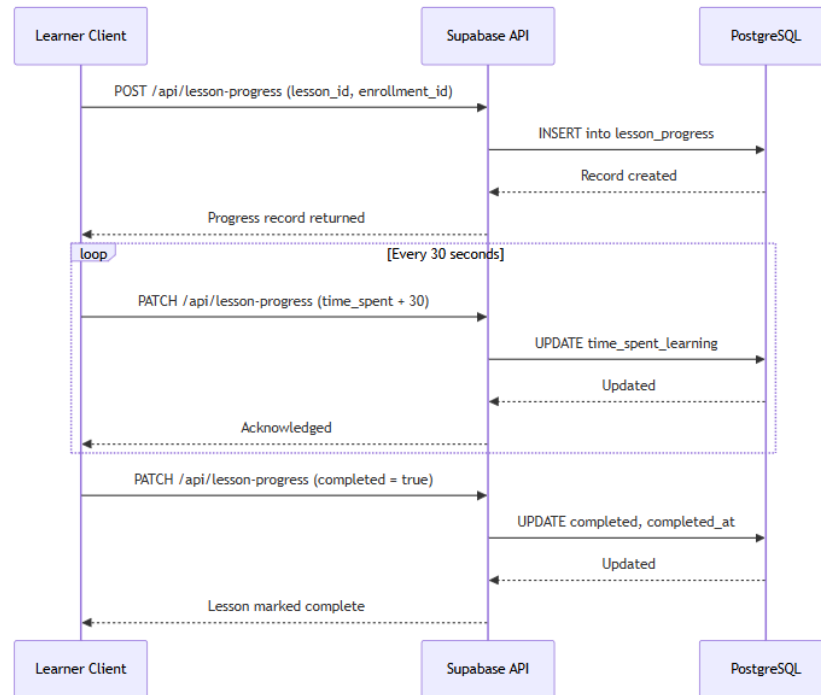


Figure 4.5: Lesson Learning Sequence Diagram

Figure 4.5 maps the continuous telemetry tracking. When a learner mounts a lesson component, an `INSERT` establishes a `lesson_progress` record. As the user dwells on the page, a client-side `setInterval` function fires every 30 seconds, dispatching an asynchronous `PATCH` request to update the `time_spent_learning` integer in the database. This guarantees highly accurate engagement metrics even if the user closes the browser abruptly.

Sequence: Quiz Attempt & Adaptive Recommendation

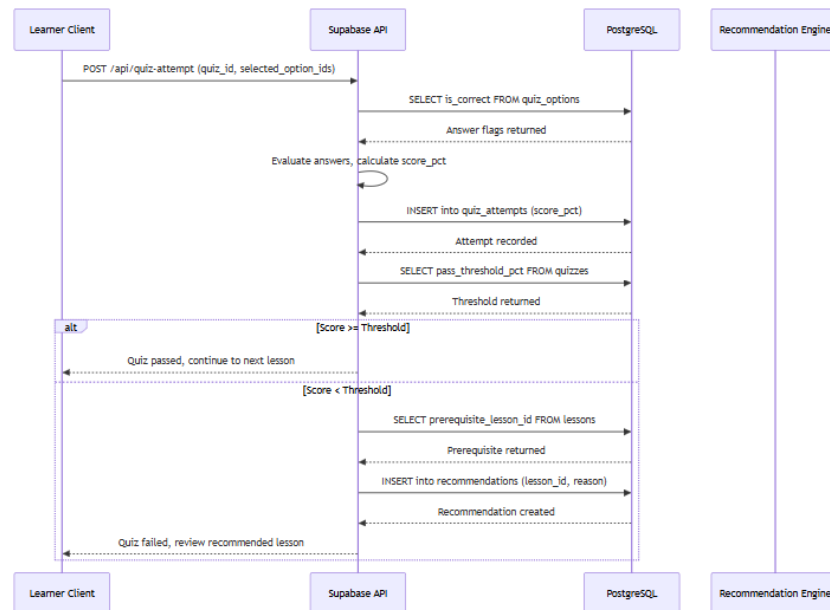


Figure 4.6: Quiz Attempt Sequence Diagram

Figure 4.6 details the most complex systemic interaction: the adaptive engine. 1. The Learner submits a JSON payload of `selected_option_id` values. 2. The backend securely evaluates these against the `is_correct` flags in `quiz_options`, ensuring the client never possesses the answer key. 3. The aggregate `score_pct` is inserted into `quiz_attempts`. 4. The server evaluates the score against `quizzes.pass_threshold_pct`. 5. If failed, the backend queries `lessons.prerequisite_lesson_id`. 6. An `INSERT` is executed on the `recommendations` table, flagging the prerequisite for mandatory review.

Sequence: Course Creation & Media Upload

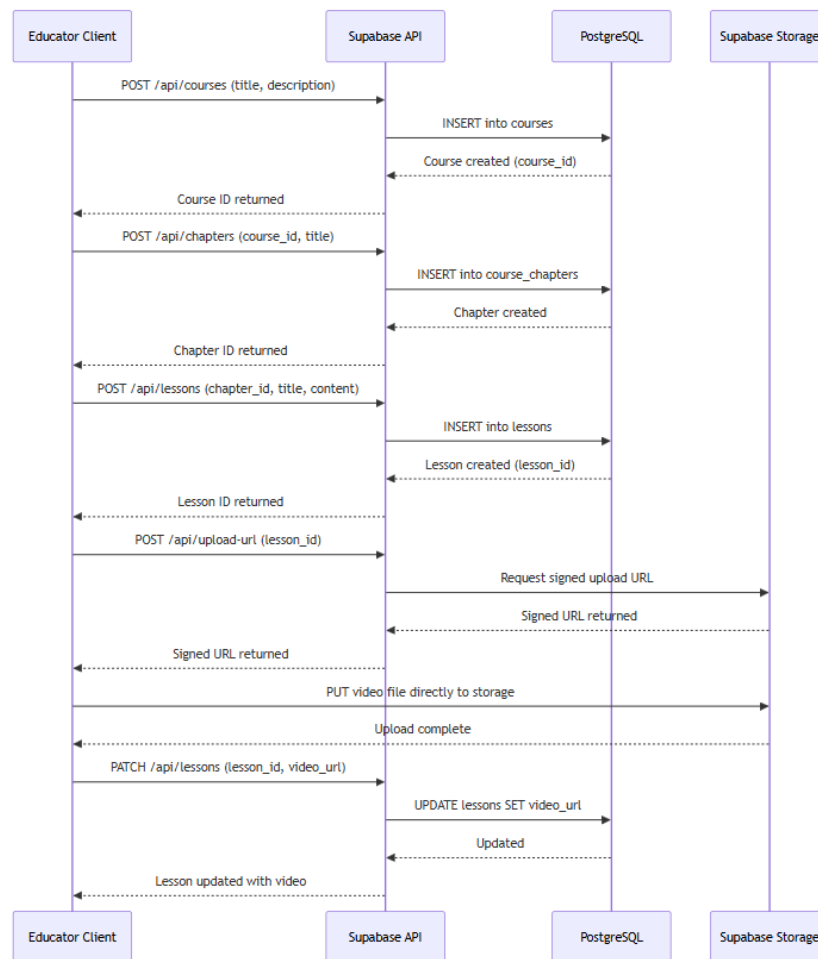


Figure 4.7: Course Creation Sequence Diagram

Figure 4.7 shows the Educator authoring pipeline. When an educator uploads a video, the Next.js API requests a signed upload URL from Supabase Storage. The client then uploads the binary blob directly to the `course-assets` bucket, bypassing the Next.js server to prevent memory bottlenecks. The resulting public URL is then stored within the `lessons.video_url` attribute.

4.2.3 User Interface Design

The interface architecture of ACCESS is organized into three role-specific shells, each with distinct navigation structures, input mechanisms, and output presentations.

Navigation Design The navigation architecture follows a role-based shell pattern enforced by Next.js middleware:

- **Public Navigation:** Unauthenticated users access the landing page, login, signup, and password reset routes. The landing page features a hero section, feature highlights, system capabilities showcase, and a role-based demo modal.
- **Learner Navigation:** The learner shell (`LearnerShell`) contains a persistent collapsible sidebar with links to Dashboard, Courses, Progress, Certificates, Achievements, and Favorites. A top bar displays the accessibility settings toggle, notification bell with unread badge count, language switcher, and user profile menu. The sidebar auto-hides in Focus Mode when navigating to lesson routes.
- **Educator Navigation:** The educator shell (`EducatorShell`) provides a sidebar with Dashboard, My Courses, All Courses, Students, Analytics, and Certificates. A course workspace sub-navigation includes tabs for Builder, Lessons, Quiz Settings, Assets, and Preview.
- **Administrator Navigation:** The admin shell (`AdminShell`) includes links to Dashboard, Users, Courses, Certificates, Analytics, Reports, Instructor Applications, and Contact Messages, with CRUD sub-navigation for course and user management.

Input Design The system implements structured input forms with client-side and server-side validation. Key input interfaces include:

- **User Registration and Profile:** Registration form collects email, password, and role selection (learner/educator) with Zod validation for email format and password strength (minimum 8 characters, uppercase, number). Profile editing allows avatar upload via Supabase Storage with MIME type validation.
- **Course Creation (Educator):** Course metadata form includes title (required, max 200 characters), slug (auto-generated with uniqueness validation), description,

difficulty level (select: beginner/intermediate/advanced), category, and thumbnail upload. Validation prevents duplicate slugs at the database level.

- **Lesson Editor (Tiptap):** Rich-text editor with constrained toolbar (headings, bold, italic, lists, links, code blocks, tables). Accessibility validation enforces heading hierarchy (no skipped levels), alt-text requirements on images, and caption requirements on video embeds. Content is sanitized server-side before persisting to `lessons.content_html`.
- **Quiz Builder (Educator):** Question creation interface supports multiple-choice and scenario types. Each question requires a minimum of two options with exactly one marked correct via radio button. Validation prevents publishing quizzes with unanswered questions or missing correct-option flags.
- **Accessibility Settings (Learner):** A modal interface with toggle switches, dropdown selects, and slider controls for font size, line spacing, theme (light/dark/soft/high_contrast), background tint picker, TTS enablement, captions, reduced motion, simplified UI, and keyboard navigation. All changes apply instantly via React Context without page reload.
- **Interactive Activity Builder (Educator):** Form-based interfaces for creating flashcards (front/back text or image), drag-and-drop activities (categories and draggable items), fill-in-the-blanks (sentence with missing word), memory games (card pairs), and timelines (ordered events). Each activity type has dedicated validation rules.

Output Design The system produces the following outputs:

- **Progress Analytics (Recharts):** Educator dashboards render bar charts, line charts, and pie charts using Recharts components, visualizing enrollment rates, lesson completion percentages, average time spent per lesson, and quiz score distributions. Data is aggregated server-side via PostgreSQL queries and delivered as typed arrays to chart components.

- **PDF Certificates (jsPDF):** Upon course completion, the system generates a landscape PDF certificate containing the learner's full name, course title, completion date, a unique alphanumeric reference code, and institutional branding. The PDF is generated server-side via `generatePDFCertificate()` in `certificate-utils.ts` and uploaded to Supabase Storage with the URL stored in the `certificates` table.
- **Administrator Reports:** Custom report generation with configurable date ranges, user filters, and metric selection. Output is available as PDF or CSV download, generated via `/api/admin/reports`.
- **Badge and Achievement Notifications:** Visual milestone badges are awarded and displayed on the learner dashboard and certificates page when predefined criteria (e.g., 100% module completion, perfect quiz score) are met, evaluated by the `achievement-engine.ts`.
- **Progress Summary:** The learner dashboard displays a progress overview card showing enrolled course count, completed lessons, certificates earned, and achievements unlocked, with a call-to-action for the next recommended lesson.

4.2.4 Database Design & Entity Relationship

The data architecture is completely normalized. Data isolation is guaranteed via Row-Level Security (RLS) policies [Supabase, 2024]. The ERD is organised into three logical domains for clarity:

1. **Identity & Configuration Domain:** Centred on the `users` table, encompassing `user_profiles`, `user_accessibility_preferences`, `enrollments`, `accessibility_templates`, `adaptive_interactions`, `instructor_applications`, `password_reset_tokens`, and `referral_codes`. This domain manages authentication, role assignment, and per-user accessibility state.

2. **Curriculum & Telemetry Domain:** Encompassing courses, lessons, quizzes, quiz_questions, quiz_options, course_chapters, course_milestones, course_achievements, quiz_attempts, quiz_answers, lesson_progress, lesson_checkpoints, learner_milestones, video_questions, recommendations, and user_achievements. This domain tracks all learning content structure and learner engagement telemetry.
3. **Content & Media Domain:** Encompassing lesson_comments, lesson_summaries, lesson_versions, lesson_interactive_content, lesson_templates, media_assets, notifications, contact_messages, certificates, certificate_templates, and certificate_verifications. This domain manages content artefacts, communication, and certification.

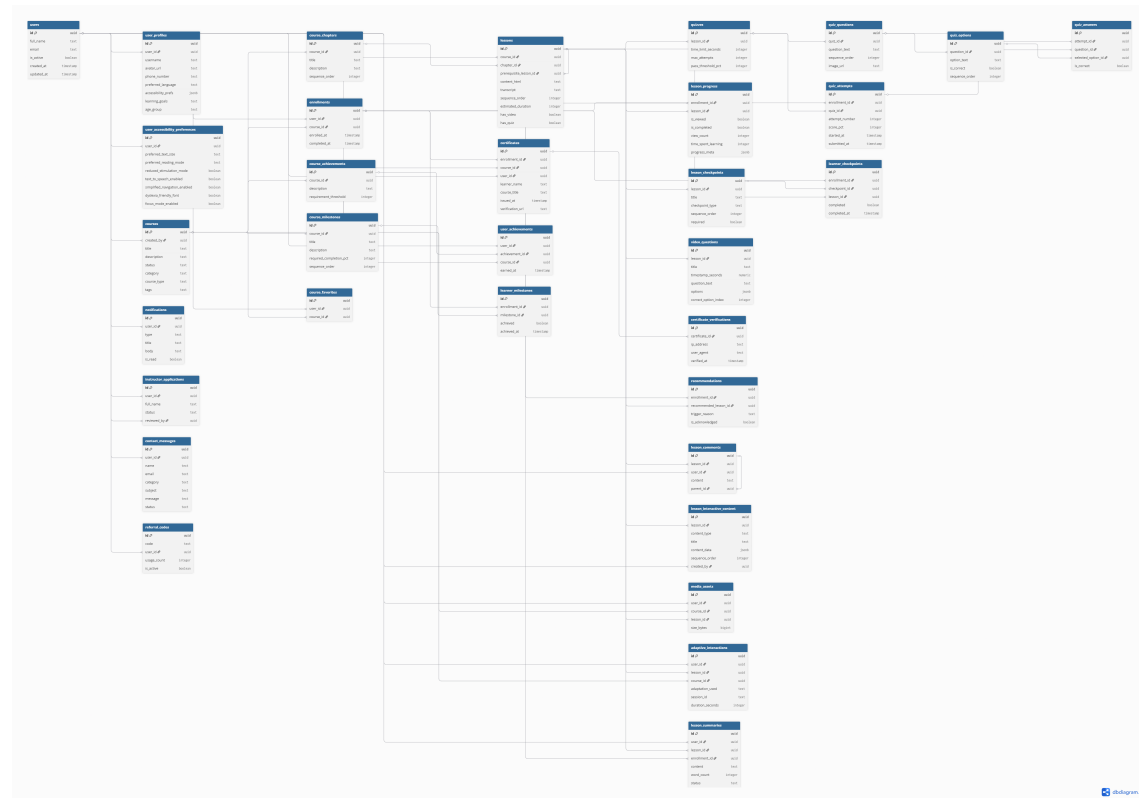


Figure 4.8 maps the complex relational lattice underpinning the system. The `users` table acts as the central hub, cascading outward to authorization profiles and created content. Due to the density of 39 interrelated tables, readers are referred to the high-resolution version in the Appendix for detailed field-level inspection.

4.3 Comprehensive Data Dictionary

The following tables constitute the exact physical schema of the ACCESS PostgreSQL database.

4.3.1 Core Identity and Configuration

1. Table: users

Table 4.1: Users Data Dictionary

Attribute	Type	Constraint	Description
id	uuid	NOT NULL	Unique identifier (UUID v4).
email	character varying	NOT NULL	User email address, used for login and notifications.
role	character varying	NOT NULL	User role: learner, educator, or admin.
is_active	boolean DEFAULT true	NOT NULL	Soft-ban flag; inactive users cannot log in.
email_verified_at	timestamp without time zone	-	Timestamp when user confirmed email via verification link; NULL if unverified.
last_login_at	timestamp without time zone	-	Timestamp of last successful login.
created_at	timestamp without time zone DEFAULT now()	NOT NULL	Timestamp when the record was created.
updated_at	timestamp without time zone DEFAULT now()	NOT NULL	Timestamp of the last modification.
deleted_at	timestamp without time zone	-	Soft-delete timestamp; null if not deleted.
full_name	text	-	User's display name shown across the platform.
instructor_application_status	text	-	Status of educator application.

2. Table: user_profiles

Table 4.2: User Profiles Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
user_id	uuid	NOT NULL	Foreign key referencing users.id.
username	text	-	Public-facing username handle.
avatar_url	text	-	URL to the user's profile image.
phone_number	text	-	Contact phone number for account recovery and notifications.
birth_date	date	-	User's date of birth for age verification and age-group analytics.
bio	text	-	Short biographical text for educator profiles.
country	text	-	ISO 3166-1 country code for regional content localisation.
preferred_language	text	DEFAULT 'en':text	UI language code (en, ms).
created_at	timestamp with time zone	DEFAULT now()	Timestamp when the record was created.
updated_at	timestamp with time zone	DEFAULT now()	Timestamp of the last modification.
learning_goals	text	-	Free-text description of learner's self-identified educational objectives.
onboarded_at	timestamp without time zone	-	Timestamp when user completed the onboarding profile setup wizard.
disability_type	character varying	-	Disability classification for adaptive engine.
accessibility_prefs	jsonb	DEFAULT ''::jsonb	JSON object storing active accessibility settings (font size, colour scheme, TTS rate, dyslexia font toggle).
notification_prefs	jsonb	DEFAULT ''::jsonb	JSON object storing notification delivery preferences (email, in-app, frequency).
age_group	text	-	Categorical age range (e.g. '13-17', '18-24') for cohort analytics.

3. Table: user_accessibility_preferences

Table 4.3: User Accessibility Preferences Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
user_id	uuid	NOT NULL	Foreign key referencing users.id.
preferred_text_size	text	DEFAULT 'medium'::text	Selected font size preference ('small', 'medium', 'large', 'x-large') applied via CSS custom properties.
preferred_reading_mode	text	DEFAULT 'standard'::text	Current reading mode layout ('standard', 'focus', 'simplified') controlling sidebar and widget visibility.
reduced_stimulation_mode	boolean DEFAULT false	NOT NULL	Boolean toggle to suppress animations, auto-playing media, and high-contrast visual elements.
text_to_speech_enabled	boolean DEFAULT false	NOT NULL	Boolean flag activating the browser-native SpeechSynthesis API for lesson content read-aloud.
simplified_navigation_enabled	boolean DEFAULT false	NOT NULL	Boolean toggle replacing complex menus with minimal, linear navigation links.
dyslexia_friendly_font	boolean DEFAULT false	NOT NULL	Switches to Atkinson Hyperlegible typeface.
focus_mode_enabled	boolean DEFAULT false	NOT NULL	Boolean flag triggering Focus Mode which hides non-essential UI elements during lessons.
created_at	timestamp with time zone DEFAULT now()	NOT NULL	Timestamp when the record was created.
updated_at	timestamp with time zone DEFAULT now()	NOT NULL	Timestamp of the last modification.
tts_rate	real	DEFAULT 1.0	Speech synthesis playback speed multiplier (0.5 to 2.0) for text-to-speech output.
tts_voice_uri	text	-	URI identifier of the selected TTS voice from the browser's SpeechSynthesisVoice list.
custom_notes	text	-	Free-text field for learner's personal annotations or study notes.

4. Table: enrollments

Table 4.4: Enrollments Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
user_id	uuid	NOT NULL	Foreign key referencing users.id.
course_id	uuid	NOT NULL	Foreign key referencing courses.id.
enrolled_at	timestamp without time zone DEFAULT now()	NOT NULL	Timestamp when the enrollment was created.
completed_at	timestamp without time zone	-	Timestamp when the record was completed.
status	character varying DEFAULT 'active'::character varying	NOT NULL	Record status (e.g., active, completed, dropped).

5. Table: accessibility_templates

Table 4.5: Accessibility Templates Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
name	text	NOT NULL	Display name of the record.
description	text	-	Free-text description or summary.
target_disability	text	NOT NULL	Disability category the template targets (e.g. 'dyslexia', 'adhd', 'asd').
content_structure	jsonb	NOT NULL	JSON definition of template layout rules and component arrangement.
created_at	timestamp with time zone DEFAULT now()	NOT NULL	Timestamp when the record was created.

6. Table: adaptive_interactions

Table 4.6: Adaptive Interactions Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
user_id	uuid	NOT NULL	Foreign key referencing users.id.
lesson_id	uuid	-	Foreign key referencing lessons.id.
course_id	uuid	-	Foreign key referencing courses.id.
adaptation_used	text	NOT NULL	Identifier of the adaptive preset applied (e.g. 'dyslexia_standard', 'adhd_focus').
session_id	text	-	Unique identifier linking interactions within a single browsing session.
duration_seconds	integer	-	Duration in seconds.
created_at	timestamp with time zone DEFAULT now()	NOT NULL	Timestamp when the record was created.

7. Table: instructor_applications

Table 4.7: Instructor Applications Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
user_id	uuid	-	Foreign key referencing users.id.
full_name	text	NOT NULL	User's display name shown across the platform.
email	text	NOT NULL	User email address, used for login and notifications.
experience	text	-	Free-text description of applicant's teaching or subject-matter experience.
reason	text	-	Applicant's motivation statement for wanting to become an educator on the platform.
portfolio_links	text	-	Comma-separated URLs to the applicant's teaching portfolio or work samples.
referral_code	text	-	Unique referral token string.
status	text DEFAULT 'pending'::text	NOT NULL	Record status (e.g., active, completed, dropped).
admin_notes	text	-	Internal reviewer notes regarding the application decision.
reviewed_by	uuid	-	UUID of the administrator who reviewed the application.
reviewed_at	timestamp with time zone	-	Timestamp of the review decision.
created_at	timestamp with time zone	DEFAULT now()	Timestamp when the record was created.
updated_at	timestamp with time zone	DEFAULT now()	Timestamp of the last modification.

8. Table: password_reset_tokens

Table 4.8: Password Reset Tokens Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
user_id	uuid	NOT NULL	Foreign key referencing users.id.
email	text	NOT NULL	User email address, used for login and notifications.
token	text	NOT NULL	Cryptographically generated unique token sent to user's email for password reset.
expires_at	timestamp with time zone	NOT NULL	Token expiration timestamp.
used	boolean DEFAULT false	NOT NULL	Boolean flag indicating whether this token has been consumed; prevents reuse.
created_at	timestamp with time zone DEFAULT now()	NOT NULL	Timestamp when the record was created.

9. Table: referral_codes

Table 4.9: Referral Codes Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
code	text	NOT NULL	Unique alphanumeric code assigned to a user for referral tracking.
user_id	uuid	-	Foreign key referencing users.id.
usage_count	integer	DEFAULT 0	Number of times used.
max_uses	integer	DEFAULT 50	Maximum number of times this referral code can be redeemed.
is_active	boolean	DEFAULT true	Soft-ban flag; inactive users cannot log in.
created_at	timestamp with time zone	DEFAULT now()	Timestamp when the record was created.

4.3.2 Curriculum Structure

10. Table: courses

Table 4.10: Courses Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
created_by	uuid	NOT NULL	Foreign key referencing the creating user.
title	character varying	NOT NULL	Display title of the record.
slug	character varying	NOT NULL	URL-safe identifier string.
description	text	-	Free-text description or summary.
status	character varying DEFAULT 'draft'::character varying	NOT NULL	Record status (e.g., active, completed, dropped).
difficulty_level	character varying	-	Course difficulty classification ('beginner', 'intermediate', 'advanced') for learner filtering.
thumbnail_url	character varying	-	URL to the thumbnail image.
published_at	timestamp without time zone	-	Timestamp when published.
created_at	timestamp without time zone DEFAULT now()	NOT NULL	Timestamp when the record was created.
updated_at	timestamp without time zone DEFAULT now()	NOT NULL	Timestamp of the last modification.
deleted_at	timestamp without time zone	-	Soft-delete timestamp; null if not deleted.
category	text	-	Content category classification.
course_type	text DEFAULT 'educator'::text	NOT NULL	Origin of course creation ('educator' or 'system') determining edit permissions.
system_course	boolean DEFAULT false	NOT NULL	Boolean flag indicating the course was created by administrators, not educators.
built_in_course	boolean DEFAULT false	NOT NULL	Boolean flag marking pre-seeded platform courses that cannot be deleted.
created_by_role	text DEFAULT 'educator'::text	NOT NULL	Role of the user who created the course ('educator' or 'admin') for permission checks.
guided_learning_enabled	boolean DEFAULT false	NOT NULL	Boolean toggle enabling step-by-step guided lesson progression.
official_course_order	integer	-	Integer display priority for system-created courses in the catalog.
managed_by_admin	boolean DEFAULT false	NOT NULL	Boolean flag indicating administrative oversight of educator-created course.
recommended_age_group	text	-	Suggested learner age range for age-appropriate content filtering.
learning_streaks_enabled	boolean DEFAULT false	NOT NULL	Boolean toggle activating daily learning streak tracking and rewards.
milestone_tracking_enabled	boolean DEFAULT false	NOT NULL	Boolean toggle enabling course milestone progress indicators.
accessibility_mode_enabled	boolean DEFAULT false	NOT NULL	Boolean flag activating accessibility-first rendering for all course content.
course_layout_type	text DEFAULT 'standard'::text	NOT NULL	Visual layout template ('standard', 'simplified') for course pages.
chapter_organization_enabled	boolean DEFAULT false	NOT NULL	Boolean toggle enabling hierarchical chapter-based lesson grouping.
certificate_enabled	boolean	DEFAULT false	Boolean toggle allowing certificate generation upon course completion.
certificate_settings	jsonb	DEFAULT ''::jsonb	JSON object storing certificate configuration (template, signature, expiry).
certification_locked	boolean	DEFAULT false	Boolean flag preventing certificate issuance until all criteria are met.
supports_tts	boolean	DEFAULT false	Boolean flag indicating whether course lessons support text-to-speech playback.
supports_transcripts	boolean	DEFAULT false	Boolean flag indicating whether video lessons include text transcripts.
supports_focus_mode	boolean	DEFAULT false	Boolean flag indicating whether

11. Table: lessons

Table 4.11: Lessons Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
course_id	uuid	NOT NULL	Foreign key referencing courses.id.
title	character varying	NOT NULL	Display title of the record.
content_html	text	-	Rich-text HTML body of the lesson rendered via Tiptap editor.
video_url	character varying	-	URL of the embedded video asset for this lesson.
transcript	text	-	Full text transcript of the lesson content for accessibility compliance.
sequence_order	integer	NOT NULL	Integer determining the display order of this lesson within its course.
status	character varying DEFAULT 'draft'::character varying	NOT NULL	Record status (e.g., active, completed, dropped).
created_at	timestamp without time zone DEFAULT now()	NOT NULL	Timestamp when the record was created.
updated_at	timestamp without time zone DEFAULT now()	NOT NULL	Timestamp of the last modification.
estimated_duration	integer	-	Approximate lesson completion time in minutes for progress estimation.
prerequisite_lesson_id	uuid	-	UUID of the lesson that must be completed before this one unlocks.
simplified_summary	text	-	Plain-text summary of lesson content for simplified reading mode.
accessibility_notes	text	-	Accessibility improvement notes.
focus_mode_enabled	boolean DEFAULT false	NOT NULL	Boolean flag allowing Focus Mode override for this specific lesson.
chunked_content_enabled	boolean DEFAULT false	NOT NULL	Boolean toggle enabling lesson content to be split into digestible segments.
lesson_type	text DEFAULT 'standard'::text	NOT NULL	Lesson classification ('standard', 'video', 'interactive') determining content rendering.
chapter_id	uuid	-	Foreign key referencing course.chapters.id.
chapter_title	text	-	Title of the chapter this lesson belongs to for hierarchical organisation.
learning_objectives	text	-	Semicolon-separated list of learning outcomes displayed before lesson content.
is_template	boolean DEFAULT false	NOT NULL	Boolean flag marking this lesson as a reusable template for creating new lessons.
template_id	uuid	-	UUID of the source template lesson this was cloned from.
scheduled_release_at	timestamp with time zone	-	Timestamp for scheduled content release (drip publishing).
visibility_status	text DEFAULT 'visible'::text	NOT NULL	Content visibility state ('visible', 'hidden', 'draft') controlling catalog display.
checkpoints_enabled	boolean DEFAULT false	NOT NULL	Boolean toggle requiring milestone checkpoints before lesson progression.
adaptive_learning_enabled	boolean DEFAULT false	NOT NULL	Boolean flag activating adaptive content delivery for this lesson.
has_video	boolean	DEFAULT true	Boolean flag indicating whether this lesson contains an embedded video.
has_pdf	boolean	DEFAULT true	Boolean flag indicating whether this lesson has a downloadable PDF resource.
has_quiz	boolean	DEFAULT true	Boolean flag indicating whether this lesson has an associated quiz.
has_transcript	boolean	DEFAULT true	Boolean flag indicating whether this lesson includes a text transcript.

12. Table: quizzes

Table 4.12: Quizzes Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
lesson_id	uuid	NOT NULL	Foreign key referencing lessons.id.
title	character varying	NOT NULL	Display title of the record.
time_limit_seconds	integer DEFAULT 0	NOT NULL	Maximum allowed quiz completion time in seconds; 0 = unlimited.
max_attempts	integer DEFAULT 0	NOT NULL	Maximum number of allowed attempts.
pass_threshold_pct	integer DEFAULT 60	NOT NULL	Minimum percentage required to pass.
created_at	timestamp without time zone DEFAULT now()	NOT NULL	Timestamp when the record was created.
updated_at	timestamp without time zone DEFAULT now()	NOT NULL	Timestamp of the last modification.

13. Table: quiz_questions

Table 4.13: Quiz Questions Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
quiz_id	uuid	NOT NULL	Foreign key referencing quizzes.id.
question_text	text	NOT NULL	Display text of the question.
question_type	character varying	NOT NULL	Question format (multiple-choice, etc.).
sequence_order	integer	NOT NULL	Integer determining the display order of this question within the quiz.
created_at	timestamp without time zone DEFAULT now()	NOT NULL	Timestamp when the record was created.
image_url	text	-	URL of an optional image asset displayed alongside the question text.

14. Table: quiz_options

Table 4.14: Quiz Options Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
question_id	uuid	NOT NULL	Foreign key referencing quiz_questions.id.
option_text	text	NOT NULL	Display text of the quiz option.
is_correct	boolean DEFAULT false	NOT NULL	Whether this is the correct answer.
sequence_order	integer	NOT NULL	Integer determining the display order of this answer option.
image_url	text	-	URL of an optional image asset displayed as this answer option.

15. Table: course_chapters

Table 4.15: Course Chapters Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
course_id	uuid	NOT NULL	Foreign key referencing courses.id.
title	text	NOT NULL	Display title of the record.
description	text	-	Free-text description or summary.
sequence_order	integer DEFAULT 0	NOT NULL	Integer determining the display order of this chapter within the course.
created_at	timestamp with time zone DEFAULT now()	NOT NULL	Timestamp when the record was created.

16. Table: course_milestones

Table 4.16: Course Milestones Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
course_id	uuid	NOT NULL	Foreign key referencing courses.id.
title	text	NOT NULL	Display title of the record.
description	text	-	Free-text description or summary.
required_completion_pct	integer DEFAULT 100	NOT NULL	Minimum percentage of lessons that must be completed to achieve this milestone.
icon	text	-	Emoji or icon identifier displayed alongside the milestone label.
sequence_order	integer DEFAULT 0	NOT NULL	Integer determining the display order of this milestone in the progress tracker.
created_at	timestamp with time zone DEFAULT now()	NOT NULL	Timestamp when the record was created.

17. Table: course_achievements

Table 4.17: Course Achievements Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
course_id	uuid	NOT NULL	Foreign key referencing courses.id.
name	character varying	NOT NULL	Display name of the record.
description	text	NOT NULL	Free-text description or summary.
icon_url	character varying	-	URL of the achievement badge image displayed in the learner's profile.
requirement_type	character varying	NOT NULL	Achievement criterion type (e.g. 'quiz_score', 'course_complete', 'streak').
requirement_threshold	integer DEFAULT 1	NOT NULL	Numeric threshold value that must be met to unlock this achievement.
created_at	timestamp with time zone DEFAULT now()	NOT NULL	Timestamp when the record was created.
updated_at	timestamp with time zone DEFAULT now()	NOT NULL	Timestamp of the last modification.

18. Table: course_accessibility_categories

Table 4.18: Course Accessibility Categories Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
course_id	uuid	NOT NULL	Foreign key referencing courses.id.
accessibility_category	text	NOT NULL	Disability category label (e.g. 'dyslexia', 'adhd', 'asd') linked to this course.
created_at	timestamp with time zone DEFAULT now()	NOT NULL	Timestamp when the record was created.

19. Table: course_favorites

Table 4.19: Course Favorites Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
user_id	uuid	NOT NULL	Foreign key referencing users.id.
course_id	uuid	NOT NULL	Foreign key referencing courses.id.
created_at	timestamp with time zone	DEFAULT now()	Timestamp when the record was created.

4.3.3 Telemetry and Assessment

20. Table: quiz_attempts

Table 4.20: Quiz Attempts Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
enrollment_id	uuid	NOT NULL	Foreign key referencing enrollments.id.
quiz_id	uuid	NOT NULL	Foreign key referencing quizzes.id.
attempt_number	integer	NOT NULL	Sequential counter for quiz attempts.
score_pct	integer	-	Quiz score as a percentage (0-100).
result	character varying DEFAULT 'in_progress':character varying	NOT NULL	Outcome of the quiz attempt ('in_progress', 'passed', 'failed').
started_at	timestamp without time zone DEFAULT now()	NOT NULL	Timestamp when the quiz attempt was initiated.
submitted_at	timestamp without time zone	-	Timestamp of response submission.

21. Table: quiz_answers

Table 4.21: Quiz Answers Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
attempt_id	uuid	NOT NULL	Foreign key referencing quiz_attempts.id.
question_id	uuid	NOT NULL	Foreign key referencing quiz_questions.id.
selected_option_id	uuid	-	Foreign key referencing the chosen quiz option.
is_correct	boolean	-	Whether this is the correct answer.

22. Table: lesson_progress

Table 4.22: Lesson Progress Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
enrollment_id	uuid	NOT NULL	Foreign key referencing enrollments.id.
lesson_id	uuid	NOT NULL	Foreign key referencing lessons.id.
is_viewed	boolean DEFAULT false	NOT NULL	Boolean flag indicating whether the learner has opened this lesson.
view_count	integer DEFAULT 0	NOT NULL	Total number of times the learner has viewed this lesson.
first_viewed_at	timestamp without time zone	-	Timestamp of the learner's first access to this lesson.
last_viewed_at	timestamp without time zone	-	Timestamp of the learner's most recent access to this lesson.
checkpoint_completed	boolean DEFAULT false	NOT NULL	Boolean flag indicating whether the learner has passed all required checkpoints.
time_spent_learning	integer DEFAULT 0	NOT NULL	Cumulative time in seconds the learner has spent on this lesson.
assisted_learning_mode	boolean DEFAULT false	NOT NULL	Boolean flag indicating whether adaptive assistance was active during this session.
summary_completed	boolean	DEFAULT false	Boolean flag indicating whether the learner submitted a lesson summary.
is_completed	boolean	DEFAULT false	Boolean flag indicating whether the learner has fully completed this lesson.
progress_meta	jsonb DEFAULT ''::jsonb	NOT NULL	JSONB object storing supplementary progress metrics and adaptive engine data.

23. Table: lesson_checkpoints

Table 4.23: Lesson Checkpoints Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
lesson_id	uuid	NOT NULL	Foreign key referencing lessons.id.
title	text	NOT NULL	Display title of the record.
description	text	-	Free-text description or summary.
checkpoint_type	text DEFAULT 'reflection'::text	NOT NULL	Checkpoint category ('reflection', 'quiz', 'activity') determining interaction format.
sequence_order	integer DEFAULT 0	NOT NULL	Integer determining the display order of this checkpoint within the lesson.
required	boolean DEFAULT false	NOT NULL	Boolean flag indicating whether this checkpoint must be passed to proceed.
created_at	timestamp with time zone DEFAULT now()	NOT NULL	Timestamp when the record was created.

24. Table: learner_checkpoints

Table 4.24: Learner Checkpoints Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
enrollment_id	uuid	NOT NULL	Foreign key referencing enrollments.id.
checkpoint_id	uuid	-	Foreign key referencing lesson_checkpoints.id.
completed	boolean DEFAULT false	NOT NULL	Boolean flag indicating whether the learner has completed this checkpoint.
completed_at	timestamp with time zone	-	Timestamp when the record was completed.
created_at	timestamp with time zone DEFAULT now()	NOT NULL	Timestamp when the record was created.
response_data	jsonb	-	JSONB payload of the learner's response.
lesson_id	uuid	-	Foreign key referencing lessons.id.

25. Table: learner_milestones

Table 4.25: Learner Milestones Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
enrollment_id	uuid	NOT NULL	Foreign key referencing enrollments.id.
milestone_id	uuid	NOT NULL	Foreign key referencing course_milestones.id.
achieved	boolean DEFAULT false	NOT NULL	Boolean flag indicating whether the learner has met this milestone.
achieved_at	timestamp with time zone	-	Timestamp when the milestone was reached.
created_at	timestamp with time zone DEFAULT now()	NOT NULL	Timestamp when the record was created.

26. Table: video_questions

Table 4.26: Video Questions Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
lesson_id	uuid	NOT NULL	Foreign key referencing lessons.id.
title	text	NOT NULL	Display title of the record.
timestamp_seconds	numeric	NOT NULL	Video playback position in seconds where the question is displayed.
question_text	text	NOT NULL	Display text of the question.
options	jsonb DEFAULT '[]'::jsonb	NOT NULL	JSONB array of answer options.
correct_option_index	integer DEFAULT 0	NOT NULL	Zero-based index of the correct answer in the options array.
sequence_order	integer DEFAULT 0	NOT NULL	Integer determining the display order of this question within the video.
created_at	timestamp with time zone DEFAULT now()	NOT NULL	Timestamp when the record was created.
updated_at	timestamp with time zone DEFAULT now()	NOT NULL	Timestamp of the last modification.

4.3.4 Learner Progress and Checkpoints

27. Table: recommendations

Table 4.27: Recommendations Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
enrollment_id	uuid	NOT NULL	Foreign key referencing enrollments.id.
recommended_lesson_id	uuid	NOT NULL	UUID of the lesson recommended to the learner by the adaptive engine.
difficulty_tier	character varying	NOT NULL	Difficulty classification of the recommended lesson relative to learner performance.
trigger_reason	text	-	Explanation of why the adaptive engine generated this recommendation.
is_acknowledged	boolean DEFAULT false	NOT NULL	Boolean flag indicating whether the learner has viewed this recommendation.
created_at	timestamp without time zone DEFAULT now()	NOT NULL	Timestamp when the record was created.

28. Table: user_achievements

Table 4.28: User Achievements Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
user_id	uuid	NOT NULL	Foreign key referencing users.id.
achievement_id	uuid	NOT NULL	Foreign key referencing course_achievements.id.
course_id	uuid	NOT NULL	Foreign key referencing courses.id.
earned_at	timestamp with time zone DEFAULT now()	NOT NULL	Timestamp when the achievement was earned.

4.3.5 Content and Media

29. Table: lesson_comments

Table 4.29: Lesson Comments Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
lesson_id	uuid	NOT NULL	Foreign key referencing lessons.id.
user_id	uuid	NOT NULL	Foreign key referencing users.id.
content	text	NOT NULL	Rich-text HTML or JSON content body.
parent_id	uuid	-	UUID of the parent comment for threaded reply nesting; NULL for top-level comments.
created_at	timestamp with time zone DEFAULT now()	NOT NULL	Timestamp when the record was created.
updated_at	timestamp with time zone DEFAULT now()	NOT NULL	Timestamp of the last modification.

30. Table: lesson_summaries

Table 4.30: Lesson Summaries Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
user_id	uuid	NOT NULL	Foreign key referencing users.id.
lesson_id	uuid	NOT NULL	Foreign key referencing lessons.id.
enrollment_id	uuid	NOT NULL	Foreign key referencing enrollments.id.
content	text	NOT NULL	Rich-text HTML or JSON content body.
word_count	integer	DEFAULT 0	Word count of the content.
status	text	DEFAULT 'draft'::text	Record status (e.g., active, completed, dropped).
ai_feedback	text	-	Automated feedback text generated by the AI summary evaluator.
educator_feedback	text	-	Manual feedback text provided by the educator on the submitted summary.
submitted_at	timestamp with time zone	-	Timestamp of response submission.
created_at	timestamp with time zone DEFAULT now()	NOT NULL	Timestamp when the record was created.
updated_at	timestamp with time zone DEFAULT now()	NOT NULL	Timestamp of the last modification.

31. Table: lesson_versions

Table 4.31: Lesson Versions Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
lesson_id	uuid	NOT NULL	Foreign key referencing lessons.id.
content_html	text	NOT NULL	Snapshot of the lesson HTML content at the time this version was saved.
version_name	text	NOT NULL	Human-readable label identifying this version (e.g. 'v1', 'draft-2024-03').
created_at	timestamp with time zone DEFAULT now()	NOT NULL	Timestamp when the record was created.
created_by	uuid	NOT NULL	Foreign key referencing the creating user.

32. Table: lesson_interactive_content

Table 4.32: Lesson Interactive Content Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
lesson_id	uuid	NOT NULL	Foreign key referencing lessons.id.
content_type	text	NOT NULL	MIME type of the content.
title	text	NOT NULL	Display title of the record.
content_data	jsonb DEFAULT ''::jsonb	NOT NULL	JSONB payload containing the H5P interactive activity configuration and state.
accessibility_settings	jsonb	DEFAULT ''::jsonb	JSON object storing accessibility overrides specific to this interactive activity.
sequence_order	integer DEFAULT 0	NOT NULL	Integer determining the display order of this activity within the lesson.
created_by	uuid	-	Foreign key referencing the creating user.
created_at	timestamp with time zone DEFAULT now()	NOT NULL	Timestamp when the record was created.
updated_at	timestamp with time zone DEFAULT now()	NOT NULL	Timestamp of the last modification.
is_draft	boolean DEFAULT false	NOT NULL	Boolean flag indicating whether this interactive content is unpublished.

33. Table: lesson_templates

Table 4.33: Lesson Templates Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
created_by	uuid	NOT NULL	Foreign key referencing the creating user.
title	text	NOT NULL	Display title of the record.
description	text	-	Free-text description or summary.
lesson_type	text DEFAULT 'standard':text	NOT NULL	Template classification ('standard', 'video', 'interactive') for the template.
content_html	text	-	Default HTML content body pre-filled in the template for educator customisation.
estimated_duration	integer	-	Default approximate completion time in minutes suggested by the template.
is_public	boolean DEFAULT false	NOT NULL	Boolean flag indicating whether this template is available to all educators.
created_at	timestamp with time zone DEFAULT now()	NOT NULL	Timestamp when the record was created.
updated_at	timestamp with time zone DEFAULT now()	NOT NULL	Timestamp of the last modification.

34. Table: media_assets

Table 4.34: Media Assets Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
user_id	uuid	NOT NULL	Foreign key referencing users.id.
course_id	uuid	-	Foreign key referencing courses.id.
file_name	character varying	NOT NULL	Original uploaded file name including extension (e.g. 'lecture.slides.pdf').
file_type	character varying	NOT NULL	MIME type of the file.
url	character varying	NOT NULL	External resource URL.
size_bytes	bigint	-	File size in bytes for storage quota enforcement.
created_at	timestamp with time zone DEFAULT now()	NOT NULL	Timestamp when the record was created.
lesson_id	uuid	-	Foreign key referencing lessons.id.
title	character varying	-	Display title of the record.

35. Table: notifications

Table 4.35: Notifications Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
user_id	uuid	NOT NULL	Foreign key referencing users.id.
type	text	NOT NULL	Classification type of the record.
title	text	NOT NULL	Display title of the record.
body	text	-	Message body text content.
metadata	jsonb	DEFAULT ''::jsonb	JSONB field for extensible metadata.
is_read	boolean	DEFAULT false	Whether the message has been read.
created_at	timestamp with time zone	DEFAULT now()	Timestamp when the record was created.

36. Table: contact_messages

Table 4.36: Contact Messages Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
name	text	NOT NULL	Display name of the record.
email	text	NOT NULL	User email address, used for login and notifications.
category	text	NOT NULL	Content category classification.
subject	text	NOT NULL	Message subject line.
message	text	NOT NULL	Full body text of the contact form submission.
status	text DEFAULT 'unread'::text	NOT NULL	Record status (e.g., active, completed, dropped).
created_at	timestamp with time zone	DEFAULT now()	Timestamp when the record was created.

37. Table: certificates

Table 4.37: Certificates Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
enrollment_id	uuid	NOT NULL	Foreign key referencing enrollments.id.
reference_code	character varying	NOT NULL	Unique alphanumeric identifier for external certificate verification.
pdf_url	character varying	-	URL to the generated PDF certificate file stored in cloud storage.
status	character varying DEFAULT 'issued':character varying	NOT NULL	Record status (e.g., active, completed, dropped).
issued_at	timestamp without time zone DEFAULT now()	NOT NULL	Timestamp when the certificate was issued.
revoked_at	timestamp without time zone	-	Timestamp when the certificate was revoked; NULL if still valid.
revoke_reason	text	-	Free-text explanation for why the certificate was revoked.
course_id	uuid	-	Foreign key referencing courses.id.
user_id	uuid	-	Foreign key referencing users.id.
learner_name	text	-	Full name of the learner as displayed on the certificate.
course_title	text	-	Title of the completed course as displayed on the certificate.
educator_name	text	-	Name of the course educator as displayed on the certificate.
institution_name	text	DEFAULT 'ACCESS Platform':text	Name of the issuing institution displayed on the certificate.
completion_date	timestamp with time zone	-	Timestamp when the learner completed all course requirements.
verification_url	text	-	Public URL for third-party verification of certificate authenticity.
skills_earned	text[]	DEFAULT '':text[]	Array of skill labels or competencies acquired through course completion.
course_duration_hours	numeric	DEFAULT 0	Total duration of the course in hours as displayed on the certificate.
signed_token	text	-	Cryptographic signature token for tamper-proof certificate verification.
template_id	text	DEFAULT 'default':text	Identifier of the certificate template used for this certificate.
metadata	jsonb	DEFAULT '':jsonb	JSONB field for extensible metadata.

38. Table: certificate_templates

Table 4.38: Certificate Templates Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
name	text	NOT NULL	Display name of the record.
description	text	-	Free-text description or summary.
layout_config	jsonb	DEFAULT ''::jsonb	JSON object defining the visual layout, positioning, and styling of certificate elements.
is_active	boolean	DEFAULT true	Soft-ban flag; inactive users cannot log in.
is_default	boolean	DEFAULT false	Boolean flag indicating this is the default certificate template used for new issuances.
created_at	timestamp with time zone	DEFAULT now()	Timestamp when the record was created.
updated_at	timestamp with time zone	DEFAULT now()	Timestamp of the last modification.

39. Table: certificate_verifications

Table 4.39: Certificate Verifications Data Dictionary

Attribute	Type	Constraint	Description
id	uuid DEFAULT gen_random_uuid()	NOT NULL	Unique identifier (UUID v4).
certificate_id	uuid	-	Foreign key referencing certificates.id.
ip_address	text	-	IP address of the client that performed the certificate verification.
user_agent	text	-	Browser user-agent string of the client that performed the verification.
verified_at	timestamp with time zone	DEFAULT now()	Timestamp when the certificate verification was performed.

4.4 Detailed Design

4.4.1 Software Design

The ACES platform follows a component-based architecture using React Server Components (RSC) and Client Components [Meta Platforms, Inc., 2023], organized by

feature domain. The key software components are described below.

Provider Components (Global State)

- **AuthProvider:** Wraps the application root. Initializes the Supabase client session via `@supabase/ssr`, exposes `user` object, `session`, `signIn()`, `signUp()`, `signOut()` through React Context. On mount, fetches the user's role and profile metadata. Responsible for session refresh and 15-minute inactivity timeout detection via `SessionTimeout` component.
- **AccessibilityProvider:** Loads `user_accessibility_settings` from Supabase on authentication. Exposes setting values and a `updateSetting()` method through context. On every setting change, writes to the database and applies CSS `data-*` attributes to the `<html>` element (e.g., `data-theme="dyslexia"`, `data-reduced-motion="true"`) which Tailwind CSS v4 [Tailwind Labs, 2023] intercepts to re-theme the entire component tree.
- **LanguageProvider:** Loads the user's `preferred_language` from profile settings, provides a `t()` function that maps keys to locale strings in `src/locales/en.ts` and `ms.ts`. Supports runtime language switching without page reload.

Learner Components

- **LearnerDashboard:** Server component that fetches enrolled courses, recommendations, recent progress, and achievement summary. Renders `WelcomeSection`, `MyCoursesSection`, `ProgressOverview`, and `QuickActions` client sub-components for interactive elements.
- **LessonPage:** Client component receiving `lessonId` as prop. Fetches lesson content (`content_html`), video URL, and quiz configuration. Renders sanitized HTML via `LessonRenderer`. In Focus Mode, conditionally omits sidebar and

header. Tracks time spent via `setInterval` dispatching `PATCH /api/lesson/[id]/progress` every 30 seconds.

- **QuizPage:** Client component managing question navigation state, timer (derived from `quizzes.time_limit_seconds`), selected answers map, and auto-submit logic on timeout. On submission, sends `selected_option_id[]` to `POST /api/quiz/[id]/attempt` and displays `QuizResultModal` with score, pass/fail status, and answer review.
- **AccessibilitySettingsModal:** Client component with toggle switches and dropdowns for all ally settings. Each input calls `updateSetting()` on the `AccessibilityProvider` context, providing real-time preview of changes.

Educator Components

- **CourseWorkspace:** Client component acting as the educator's main working area. Manages tab state (Builder, Lessons, Quiz Settings, Assets, Preview). Contains nested routing for lesson editing within the course context.
- **LessonEditor:** Client component wrapping the Tiptap editor instance [Brainhub, 2024]. Configures the editor with extensions: StarterKit, Underline, Link, Image, Table, CodeBlockLowlight, TextAlign, Highlight, TextStyle, Color, and Placeholder. Strips disallowed tags on save. Calls `PUT /api/educator/lessons/[id]` to persist the sanitized HTML.
- **QuizBuilderModal:** Client component for creating and editing quiz questions. Manages a dynamic list of questions each with a type selector (multiple-choice or scenario), question text input, and a dynamic list of option rows with radio-button correct-flag selection. Validates that each question has at least one correct option before allowing save.
- **InteractiveActivityBuilder:** Client component with sub-builders for each activity type: `FlashcardBuilder`, `DragDropBuilder`, `FillBlanksBuilder`, `MemoryGameBuilder`, `TimelineBuilder`. Each sub-builder manages its specific data structure and emits a typed JSON object on save.

Administrator Components

- **UserManagement:** Client component with a plain HTML table for sorting, filtering, and paginating the users list. Supports role reassignment via `RoleEditModal` and soft-delete toggling.
- **CourseManagement:** Client component listing all courses with status filters (draft, pending_review, published). Includes `ThumbnailPicker` for assigning stock images and `AdminCourseSettingsTab` for system-wide course configuration.
- **AnalyticsDashboard:** Client/server hybrid component. Server-fetches aggregated platform metrics (total users, enrollments, active courses, completion rates). Renders custom CSS-based bar charts with interactive tooltips and date range filtering.

Core API Routes and Server Logic

- **POST /api/recommendations/generate:** Invoked server-side after quiz attempt evaluation. Executes the `recommendation-engine.ts` which analyzes the learner's enrollment context, quiz score, and prerequisite lesson chain to generate up to three recommendations (revision, standard, advanced, or cross-course). Uses Supabase service role to bypass RLS for cross-user queries.
- **PUT /api/educator/lessons/[id]:** Receives sanitized HTML from Tiptap, validates with Zod, updates the `lessons` table, and creates a version snapshot in `lesson_versions` for rollback support.
- **POST /api/certificates/custom:** Receives a custom PDF certificate file from the educator, validates file type (PDF only) and size (max 5MB), uploads to Supabase Storage `course-assets` bucket, and stores the URL reference.
- **Recommendation Engine (`recommendation-engine.ts`):** Contains `generateRecommendations()` function. Logic branches: revision (if recent quiz failed), standard (if unviewed lessons remain), advanced (if near course

completion), cross-course (if tag-matched courses exist). Returns an ordered array of recommended lesson IDs with trigger reasons.

- **Achievement Engine (`achievement-engine.ts`):** Contains `evaluateAchievements()` function called after quiz attempts and lesson completions. Checks learner's current progress against `course_achievements` criteria and grants new badges via `user_achievements` insertion.
- **Adaptive Engine (`adaptive-engine.ts`):** Applies preset-based accessibility configuration (Dyslexia, ADHD, Autism, Dyscalculia) with age-adapted sub-variants. Each preset defines a combination of settings for font family, spacing, theme, tint, reading spotlight, focus mode, chunked content, TTS, and simplified UI, applied via CSS data attributes.

Recommendation Engine Pseudocode Algorithm 1 presents the pseudocode for the `generateRecommendations()` function, illustrating the rule-based branching logic used to generate personalized lesson recommendations.

Algorithm 1 generateRecommendations()

Require: *learner_id, course_id, quiz_score*
Ensure: Ordered list of recommended lesson IDs with reasons

```

1: enrolled  $\leftarrow$  CHECKENROLLMENT(learner_id, course_id)
2: recent_quiz  $\leftarrow$  GETRECENTQUIZATTEMPT(learner_id, course_id)
3: viewed  $\leftarrow$  GETVIEWEDLESSONS(learner_id, course_id)
4: all_lessons  $\leftarrow$  GETCOURSELESSONS(course_id)
5: recommendations  $\leftarrow$   $\emptyset$ 
6: if recent_quiz.score < passing_threshold then
7:   failing_lesson  $\leftarrow$  recent_quiz.lesson_id
8:   APPEND(recommendations, {revision, failing_lesson})
9: end if
10: for all lesson in all_lessons \ viewed do
11:   APPEND(recommendations, {standard, lesson})
12:   if |recommendations|  $\geq$  3 then
13:     break
14:   end if
15: end for
16: if course_completion_pct  $\geq$  80% then
17:   next_course  $\leftarrow$  GETTAGMATCHEDCOURSE(course_id)
18:   APPEND(recommendations, {cross-course, next_course})
19: end if
20: return recommendations

```

Data Access Layer Data fetching is abstracted into three role-specific modules:

- **learner-api.ts:** 48 exported functions covering profile management, course discovery, enrollment, lesson content retrieval, quiz attempts, progress tracking, certificate viewing, recommendation fetching, favorites, and accessibility settings CRUD.
- **educator-api.ts:** 73 exported functions for course CRUD, lesson CRUD with versioning, quiz builder operations, asset upload, student progress queries, analytics aggregation, certificate management, and instructor applications.
- **admin-api.ts:** 40 exported functions for user management, system course creation with chapters/milestones/templates, certificate oversight, instructor application review, contact message management, report generation, and platform health metrics.

4.4.2 Physical Database Design

The ACCESS database is implemented on Supabase PostgreSQL 15 [Supabase, 2023]. The physical schema enforces relational integrity through foreign keys, unique constraints, and Row-Level Security policies.

Table Definitions (DDL) Core tables are created with the following representative DDL:

users table:

```
CREATE TABLE public.users ( id UUID PRIMARY KEY DEFAULT
gen-random-uuid(), email VARCHAR(255) UNIQUE NOT NULL, full_name TEXT
NOT NULL, role VARCHAR(20) NOT NULL DEFAULT 'learner' CHECK (role IN
('learner', 'educator', 'admin')), is_active BOOLEAN DEFAULT true,
email_verified_at TIMESTAMPTZ, created_at TIMESTAMPTZ DEFAULT NOW(),
deleted_at TIMESTAMPTZ );
```

enrollments table with unique constraint:

```
CREATE TABLE public.enrollments ( id UUID PRIMARY KEY DEFAULT
gen-random-uuid(), user_id UUID NOT NULL REFERENCES users(id) ON
DELETE CASCADE, course_id UUID NOT NULL REFERENCES courses(id) ON
DELETE CASCADE, status VARCHAR(20) DEFAULT 'active' CHECK (status IN
('active', 'completed', 'dropped')), enrolled_at TIMESTAMPTZ DEFAULT
NOW(), completed_at TIMESTAMPTZ, UNIQUE(user_id, course_id) );
```

Indexes The following indexes are defined to optimize query performance:

```
CREATE INDEX idx_enrollments_user ON enrollments(user_id); CREATE
INDEX idx_enrollments_course ON enrollments(course_id); CREATE INDEX
idx_lesson_progress_enrollment ON lesson_progress(enrollment_id);
CREATE INDEX idx_quiz_attempts_enrollment ON quiz_attempts(enrollment_id);
CREATE INDEX idx_lessons_course ON lessons(course_id); CREATE
INDEX idx_notifications_user ON notifications(user_id); CREATE
```

```
INDEX idx_courses_created_by ON courses(created_by); CREATE INDEX
idx_courses_status ON courses(status);
```

Row-Level Security Policies RLS is enabled on all tables containing user data. Representative policies include:

```
-- Learners can only read their own lesson progress CREATE POLICY
lesson_progress_learner_select ON lesson_progress FOR SELECT USING
(auth.uid() IN ( SELECT user_id FROM enrollments WHERE id =
enrollment_id ));
-- Educators can CRUD their own courses CREATE POLICY
courses_educator_insert ON courses FOR INSERT WITH CHECK (auth.uid() =
created_by);
-- Admins can read all rows CREATE POLICY users_admin_select ON users
FOR SELECT USING (auth.role() = 'admin');
```

Database Triggers Five notification triggers are implemented as PL/pgSQL functions attached to INSERT and UPDATE events:

```
CREATE FUNCTION notify_on_quiz_attempt() RETURNS TRIGGER AS $$ BEGIN
IF NEW.score_pct < (SELECT pass_threshold_pct FROM quizzes WHERE id
= NEW.quiz_id) THEN INSERT INTO notifications (user_id, type, title,
body, metadata) VALUES ( (SELECT created_by FROM courses WHERE id =
( SELECT course_id FROM enrollments WHERE id = NEW.enrollment_id )),
'quiz_failed', 'Student Failed Quiz', 'A learner scored below the
passing threshold.', jsonb_build_object('attempt_id', NEW.id, 'score',
NEW.score_pct) ); END IF; RETURN NEW; END; $$ LANGUAGE plpgsql;
CREATE TRIGGER notify_on_quiz_attempt_trigger AFTER INSERT ON
quiz_attempts FOR EACH ROW EXECUTE FUNCTION notify_on_quiz_attempt();
```

4.4.3 Systemic Implementation Details

Interactive Video Questions To prevent passive consumption, ACCESS implements embedded video questions. This is achieved by creating a custom React hook

`useVideoTime()` connected to an HTML5 video player reference. An array of timestamp objects is loaded from the database. When the video's current time intersects a target timestamp, the state `isVideoPaused` is forced to true, and a z-index overlay modal is mounted containing a specific `quiz_question`. The video cannot be resumed until the user interacts with the prompt.

Database Trigger Architecture (Notifications) To maintain a decoupled architecture and prevent the Next.js API routes from becoming bloated with notification logic, ACCESS utilizes native PostgreSQL Triggers. For example, the `notify_on_quiz_attempt` trigger is attached to the `quiz_attempts` table. Upon an `INSERT` where `score_pct < pass_threshold`, the trigger executes a PL/pgSQL function that automatically formats a payload and executes an `INSERT` into the `notifications` table, targeting the `created_by` ID of the parent course, instantly alerting the educator of the failure via real-time database subscription.

Session Timeout and Security A `SessionTimeout` component wraps authenticated routes. It tracks user activity via `mousemove` and `keydown` event listeners. After 15 minutes of inactivity, a warning modal appears with a 30-second countdown. If no response is received, the user is forcibly signed out via `supabase.auth.signOut()` and redirected to the login page.

4.5 Limitations of Design

The current ACCESS design has several acknowledged limitations that define the boundary of this implementation:

- **Content scope:** The system currently supports only text-based lessons with embedded video. Audio-only content, interactive simulations, and downloadable PDF resources are not yet supported.
- **Adaptive engine:** The rule-based recommendation engine operates on quiz

performance only. It does not incorporate learner behaviour analytics such as time-on-task or scroll patterns, nor does it utilize real-time physiological signals.

- **Expert review only:** Accessibility evaluation was conducted through structured walkthroughs and expert review against WCAG 2.1 Level AA criteria. Full user testing with neurodivergent participants was beyond the scope of this project phase and is recommended for future work.
- **Single-language support:** The platform interface and content are English-only. Multilingual and right-to-left language support has not been implemented.
- **Offline capability:** All features require an active internet connection. No offline content caching or progressive web app (PWA) functionality has been implemented.

4.6 Summary

This chapter translated the project's requirements into concrete engineering designs. The High-Level Design section presented behavioral activity diagrams and interaction sequence diagrams mapping critical system workflows, followed by a structured user interface design covering navigation patterns, input forms with validation rules, and output specifications. The conceptual and logical database design was documented through an Entity Relationship Diagram and an exhaustive data dictionary. The Detailed Design section specified the key software components, including provider contexts, role-specific React components, API routes, and core server engines, alongside the physical database implementation with DDL statements, indexes, Row-Level Security policies, and database triggers. The systemic implementation details described the interactive video question system, notification trigger architecture, and session timeout security mechanism.

References

- [Anthology Inc., 2023] Anthology Inc. (2023). Blackboard learn.
- [Atkinson, 2021] Atkinson, J. B. (2021). Atkinson hyperlegible font. *Braille Institute of America*.
- [Barkley, 1997] Barkley, R. A. (1997). Implications of executive function deficits for treatment of adhd children: Empirical and practical issues. *Journal of Clinical Child Psychology*, 26(2):129–140.
- [Batanero et al., 2014] Batanero, C., Boticario, J. G., et al. (2014). Accessibility of learning management systems: A review. *Journal of Universal Computer Science*, 20(9):1253–1272.
- [Brainhub, 2024] Brainhub (2024). Tiptap: The headless rich text editor framework.
- [Burl and O’Brien, 2024] Burl, M. and O’Brien, K. (2024). Dropout rates and accessibility barriers in e-learning for neurodivergent students. *Computers & Education*, 198:104–118.
- [Figma, 2023] Figma (2023). Figma: The collaborative interface design tool.
- [Framer, 2024] Framer (2024). Framer motion: Production-ready motion library for react.
- [Franceschini et al., 2013] Franceschini, S., Gori, S., Ruffino, M., Viola, S., Molteni, M., and Facoetti, A. (2013). Action video games make dyslexic children read better. *Current Biology*, 23(6):462–466.
- [Google LLC, 2023] Google LLC (2023). Google classroom.
- [Instructure, Inc., 2023] Instructure, Inc. (2023). Canvas learning management system.
- [Lawson, 2011] Lawson, W. (2011). *The Passionate Mind: How People with Autism Learn*. Jessica Kingsley Publishers.
- [Meta Platforms, Inc., 2023] Meta Platforms, Inc. (2023). React: The library for web and native user interfaces.

- [Meyer et al., 2014] Meyer, A., Rose, D. H., and Gordon, D. (2014). *Universal Design for Learning: Theory and Practice*. CAST Professional Publishing.
- [Microsoft, 2023] Microsoft (2023). Typescript: Javascript with syntax for types.
- [MoodleHQ, 2023a] MoodleHQ (2023a). Moodle architecture: Technical overview.
- [MoodleHQ, 2023b] MoodleHQ (2023b). Moodle: Open-source learning platform.
- [Munoz et al., 2003] Munoz, D. P., Armstrong, I. T., Hampton, K. A., and Moore, K. D. (2003). Saccadic eye movements in attention deficit hyperactivity disorder. *Clinical Neuropsychology*, 114(9):1667–1674.
- [Prensky, 2001] Prensky, M. (2001). Digital natives, digital immigrants part 1. *On the Horizon*, 9(5):1–6.
- [Recharts Team, 2024] Recharts Team (2024). Recharts: A composable charting library built on react components.
- [Rello and Baeza-Yates, 2013] Rello, L. and Baeza-Yates, R. (2013). Good fonts for dyslexia. In *Proceedings of the 15th International ACM SIGACCESS Conference on Computers and Accessibility*, pages 1–8.
- [Schwaber, 1997] Schwaber, K. (1997). Scrum development process. In *Business Object Design and Implementation: OOPSLA’95 Workshop Proceedings*, pages 117–134. Springer.
- [Spencer and Podesta, 2022] Spencer, M. and Podesta, F. (2022). Neurodiversity in higher education: A comprehensive review of progress and challenges. *Journal of Applied Higher Education*, 13(2):45–62.
- [Supabase, 2023] Supabase (2023). Supabase documentation: The open source firebase alternative.
- [Supabase, 2024] Supabase (2024). Supabase row level security documentation.
- [Sweller, 1988] Sweller, J. (1988). Cognitive load during problem solving: Effects on learning. *Cognitive science*, 12(2):257–285.
- [Tailwind Labs, 2023] Tailwind Labs (2023). Tailwind css documentation.

[Vercel Inc., 2023a] Vercel Inc. (2023a). Next.js documentation: Server-side rendering (ssr).

[Vercel Inc., 2023b] Vercel Inc. (2023b). Vercel documentation: The platform for frontend developers.

[W3C Web Accessibility Initiative, 2023] W3C Web Accessibility Initiative (2023). Web content accessibility guidelines (wcag) 2.2.

[Zelazo et al., 2003] Zelazo, P. D., Muller, U., Frye, D., and Marcovitch, S. (2003). The development of executive function in early childhood. *Monographs of the Society for Research in Child Development*, 68(3):vii–137.